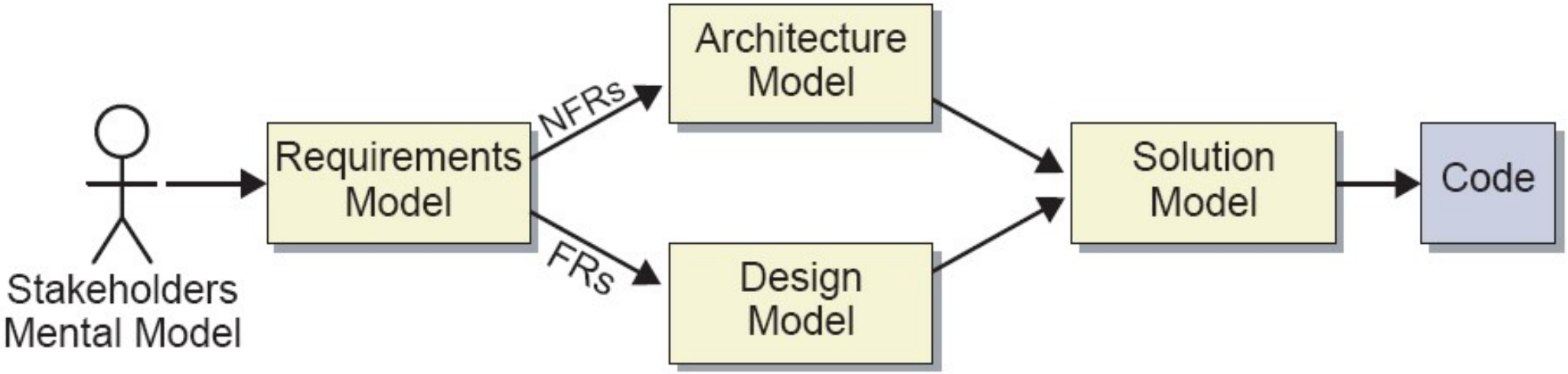
Objektno orijentirani razvoj

2025/26.06

CC BY NC SA

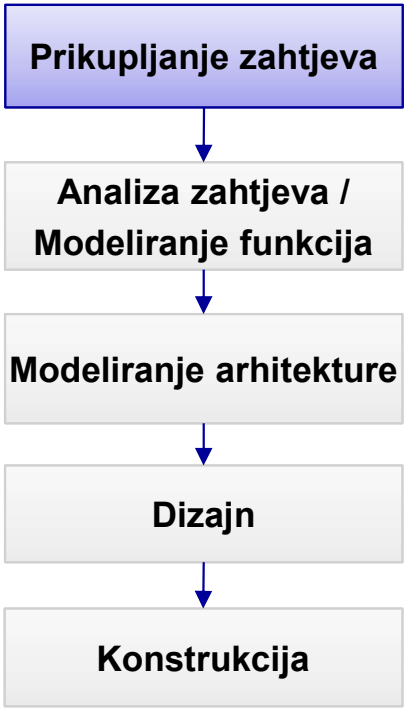
Objektno orijentirani razvoj softvera

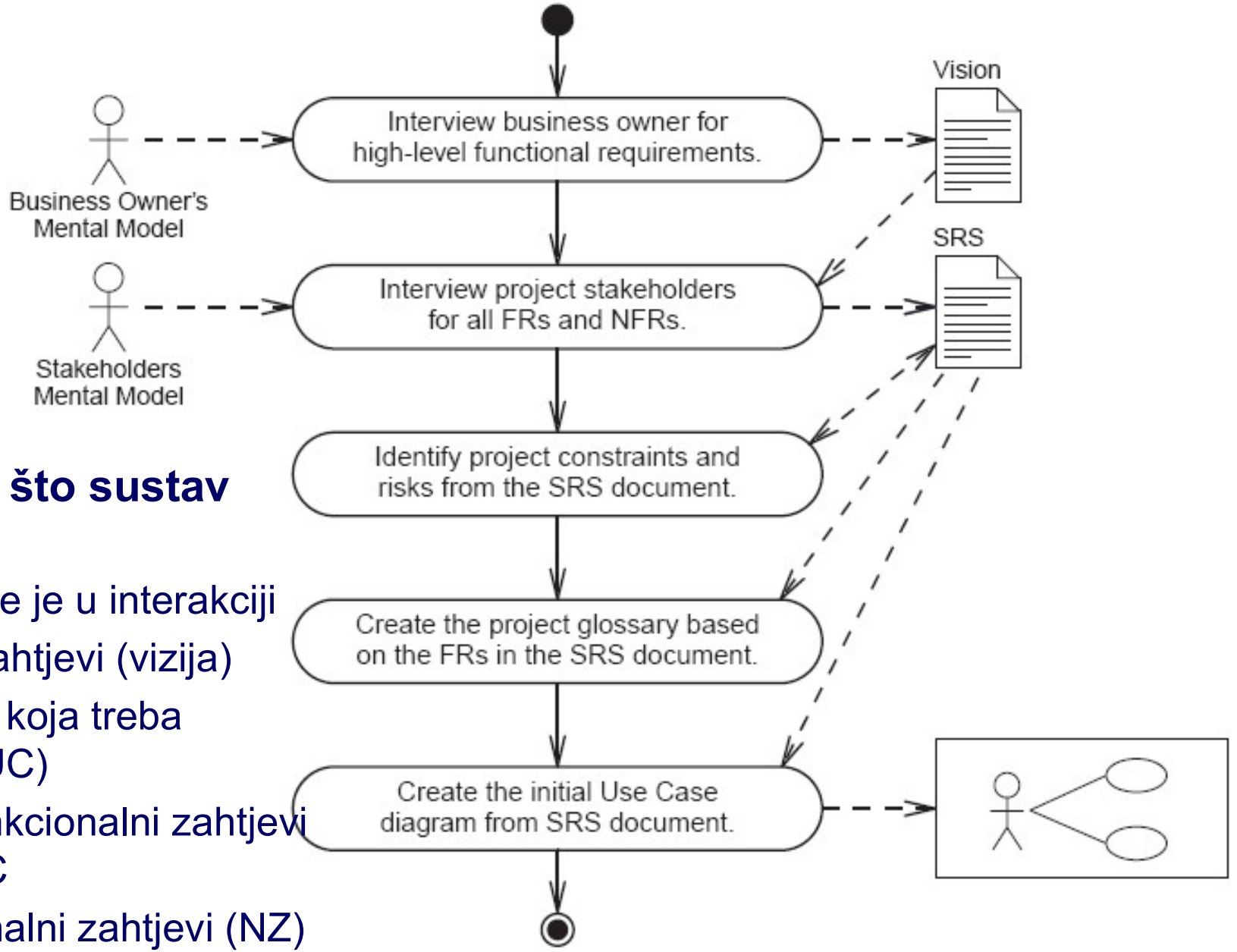
- niz slijednih transformacija od modela do koda
- NFRs – programskim okvirima, ponovno iskoristivim knjižicama, ...
- FRs – projektiranjem prema zahtjevima korisnika



© Object-Oriented Analysis and Design Using UML
Sun Microsystems, Inc.

Radni tok prikupljanja zahtjeva





- **Određivanje što sustav treba činiti**
  - s kime/čime je u interakciji
  - poslovni zahtjevi (vizija)
  - ponašanja koja treba podržati (UC)
  - detaljni funkcionalni zahtjevi (FZ) za UC
  - nefunkcionalni zahtjevi (NZ)

Prikupljanje zahtjeva

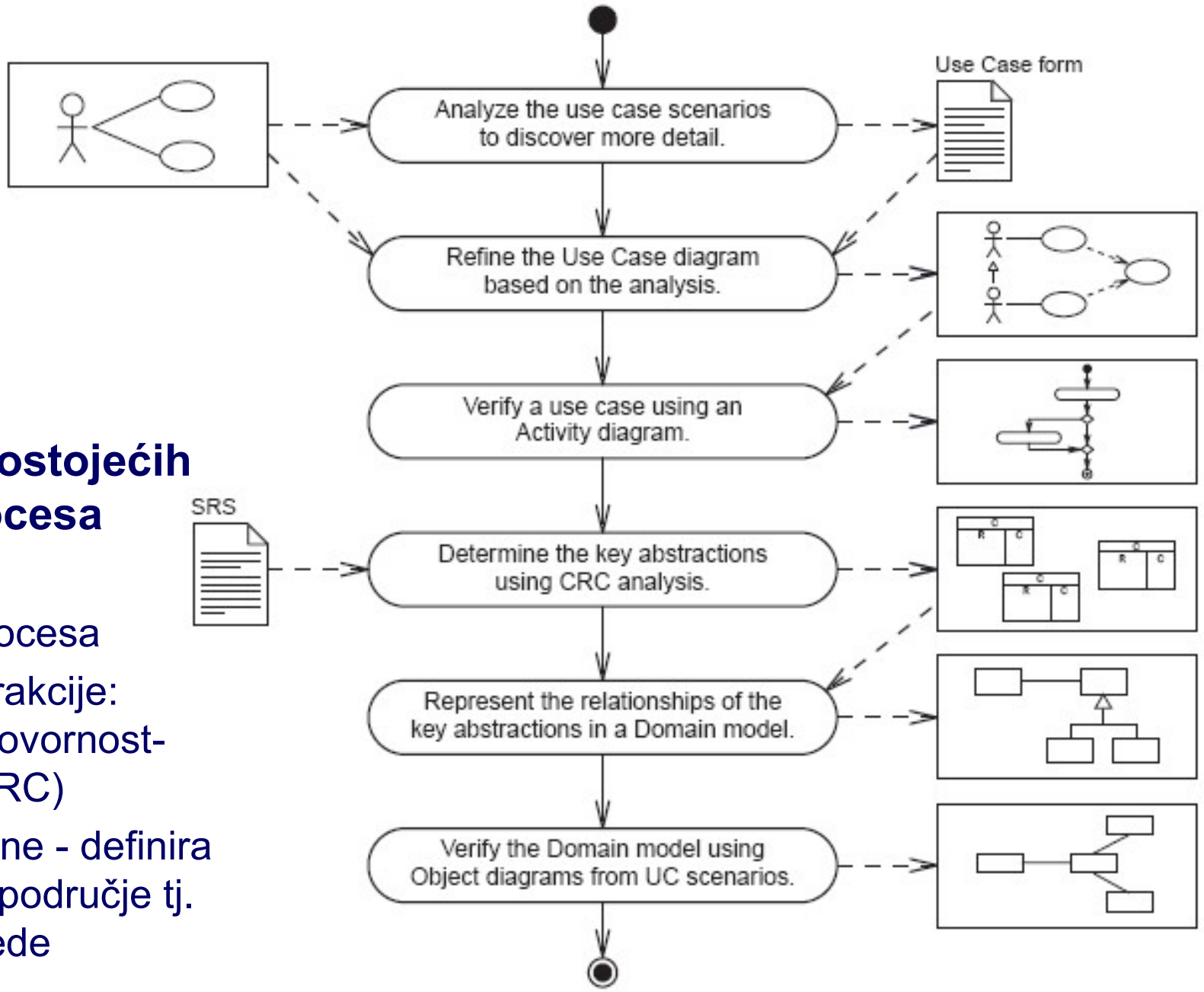
Analiza zahtjeva / Modeliranje funkcija

Modeliranje arhitekture

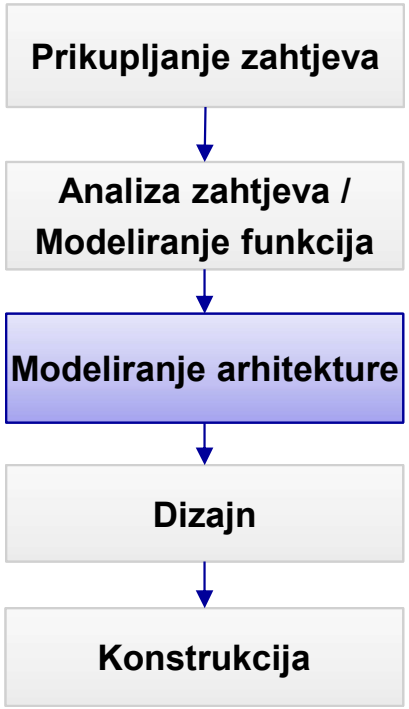
Dizajn

Konstrukcija

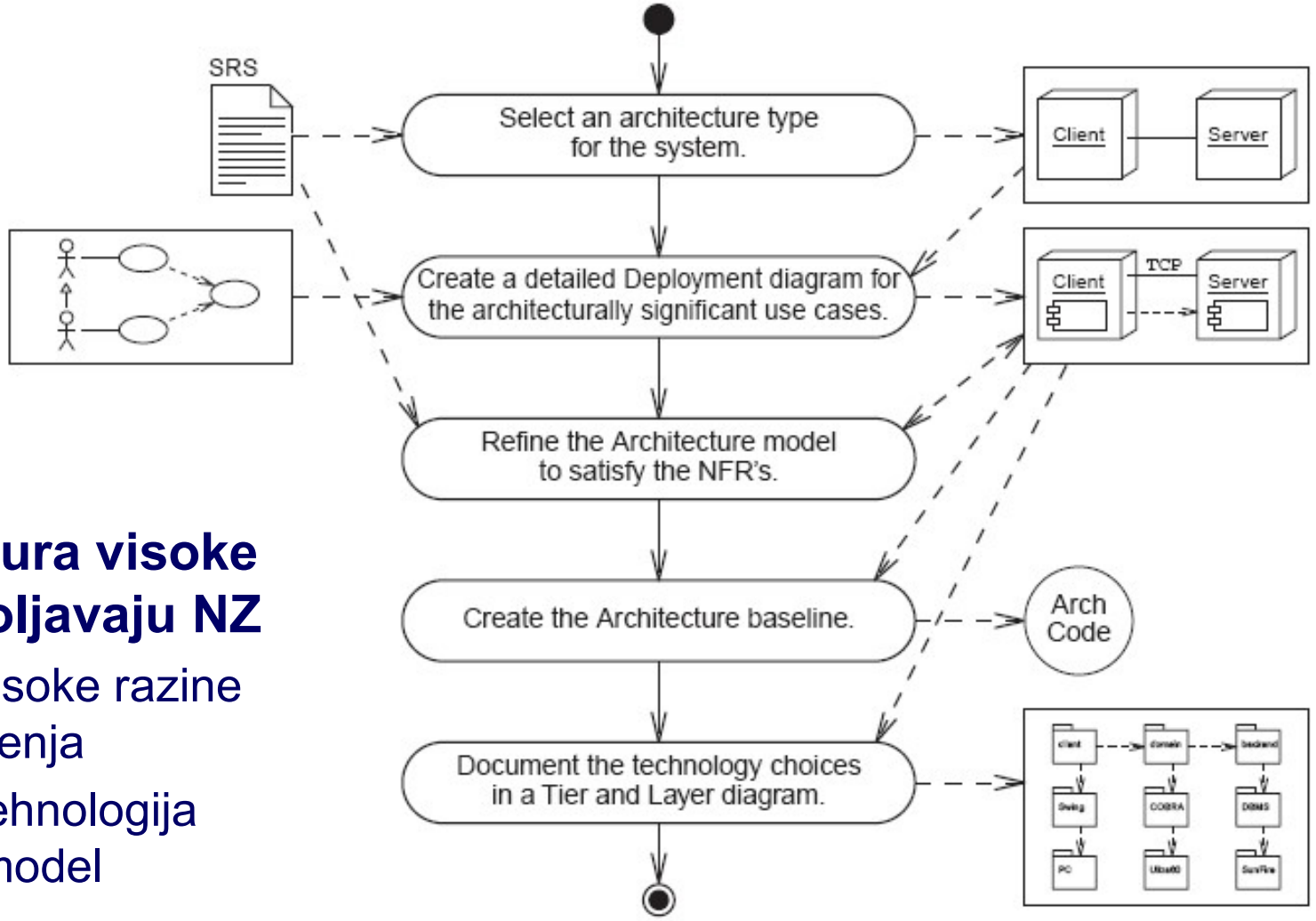
Radni tok analize zahtjeva (→ modeliranje funkcija)



- **Modeliranje postojećih poslovnih procesa**
  - rafinirani UC
  - aktivnosti procesa
  - ključne apstrakcije: Razred-Odgovornost-Suradnja (CRC)
  - model domene - definira problemsko područje tj. ključne razrede

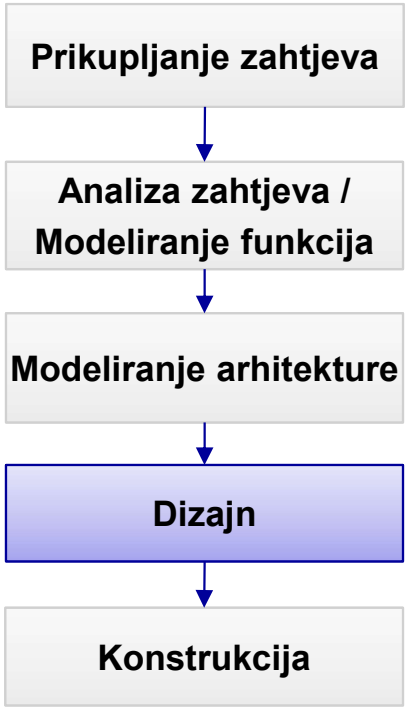


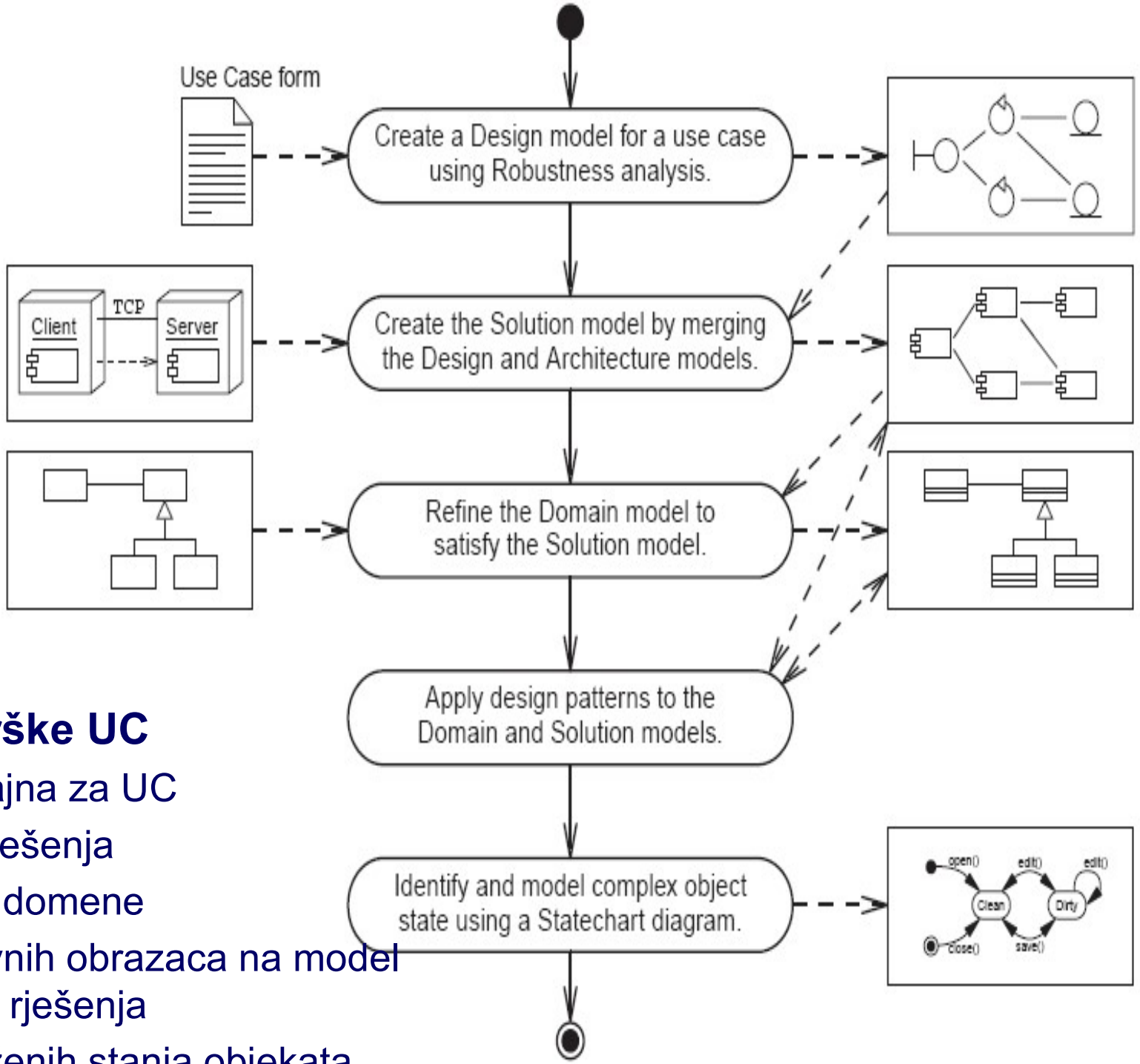
Radni tok modeliranja arhitekture



- **Modeliranje struktura visoke razine koje zadovoljavaju NZ**
  - razvoj struktura visoke razine programskog rješenja
  - ustanovljavanje tehnologija koje podržavaju model arhitekture
  - pojašnjenje modela arhitekture obrascima arhitekture koji zadovoljavaju NZ

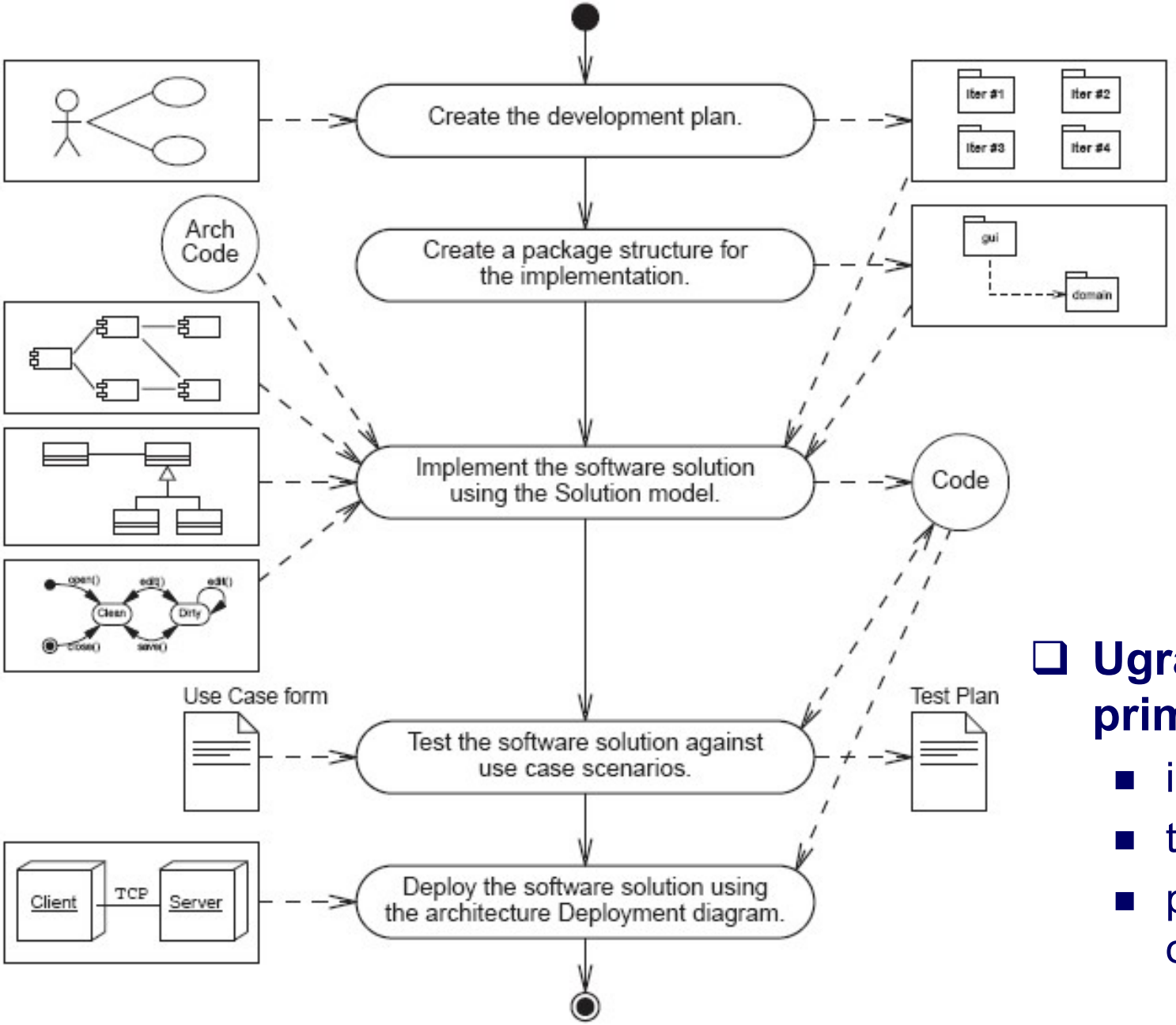
Radni tok dizajna

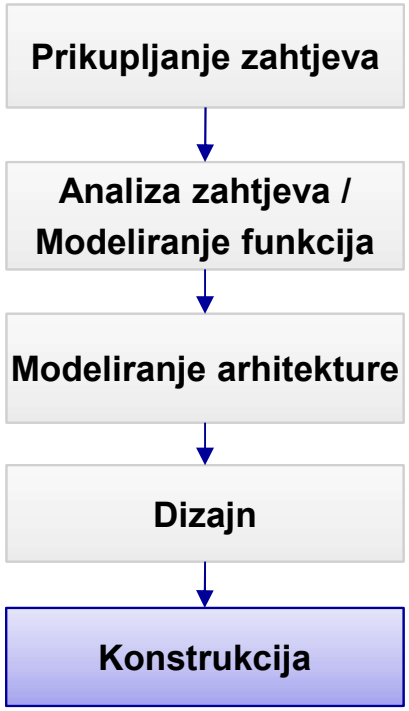




- **Modeliranje podrške UC**
  - modeliranje dizajna za UC
  - izrada modela rješenja
  - razrada modela domene
  - primjena oblikovnih obrazaca na model domene i model rješenja
  - modeliranje složenih stanja objekata

Radni tok konstrukcije





- **Ugradnja, provjera i primjena sustava**
  - implementacija rješenja
  - testiranje
  - primjena u radnom okruženju

Modeliranje procesa i funkcionalnosti

Business process and functional modeling

cc BY NC SA

Modeliranje funkcionalnosti

- **Functional modeling**
  - Opisuje poslovne procese i interakciju IS s okolinom
  - Dokumentiranje zahtjeva i vanjskog ponašanja sustava
  - Određivanje potencijalnih (logičkih) sučelja sustava (prema okolini)

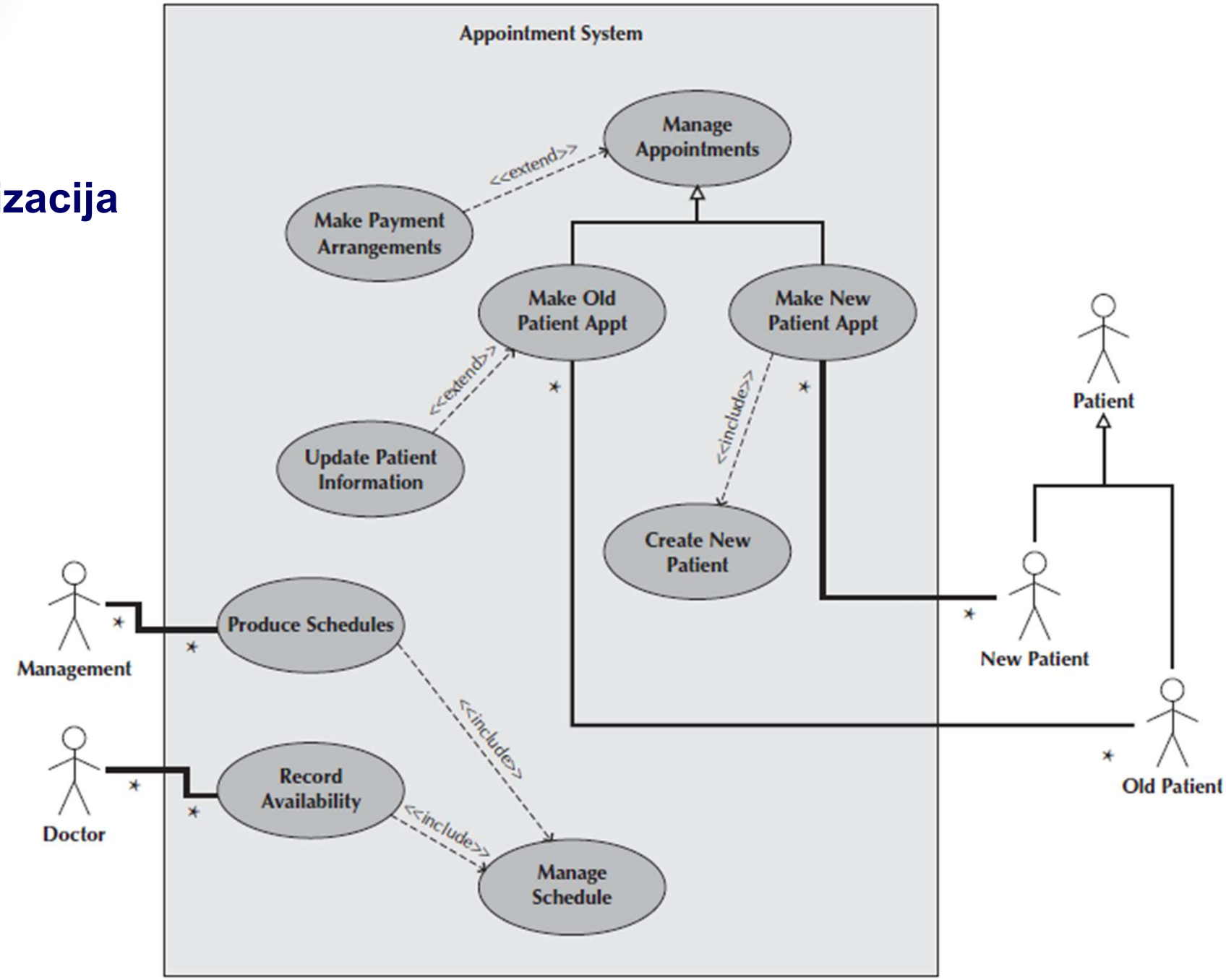
- **Logički, modeli problemskog područja (problem domain models)**
  - opis aktivnosti bez detalja provedbe
  - ne mora se znati izvode li se računalom ili ručno !
  - **Slučaj korištenja**
    - prikaz aktivnosti korisnika
    - vanjski ili funkcionalni pogled na IS – osnovne funkcije / interakcija sustava
  - **Dijagram aktivnosti**
    - logičko modeliranje poslovnih procesa i tokova rada
    - prikaz procesa ili aktivnosti te kolanja podataka

- **Fizički modeli**
  - temeljem logičkih modela za vrijeme faze oblikovanja
  - informacije potrebne za izgradnju sustava

cc BY NC SA

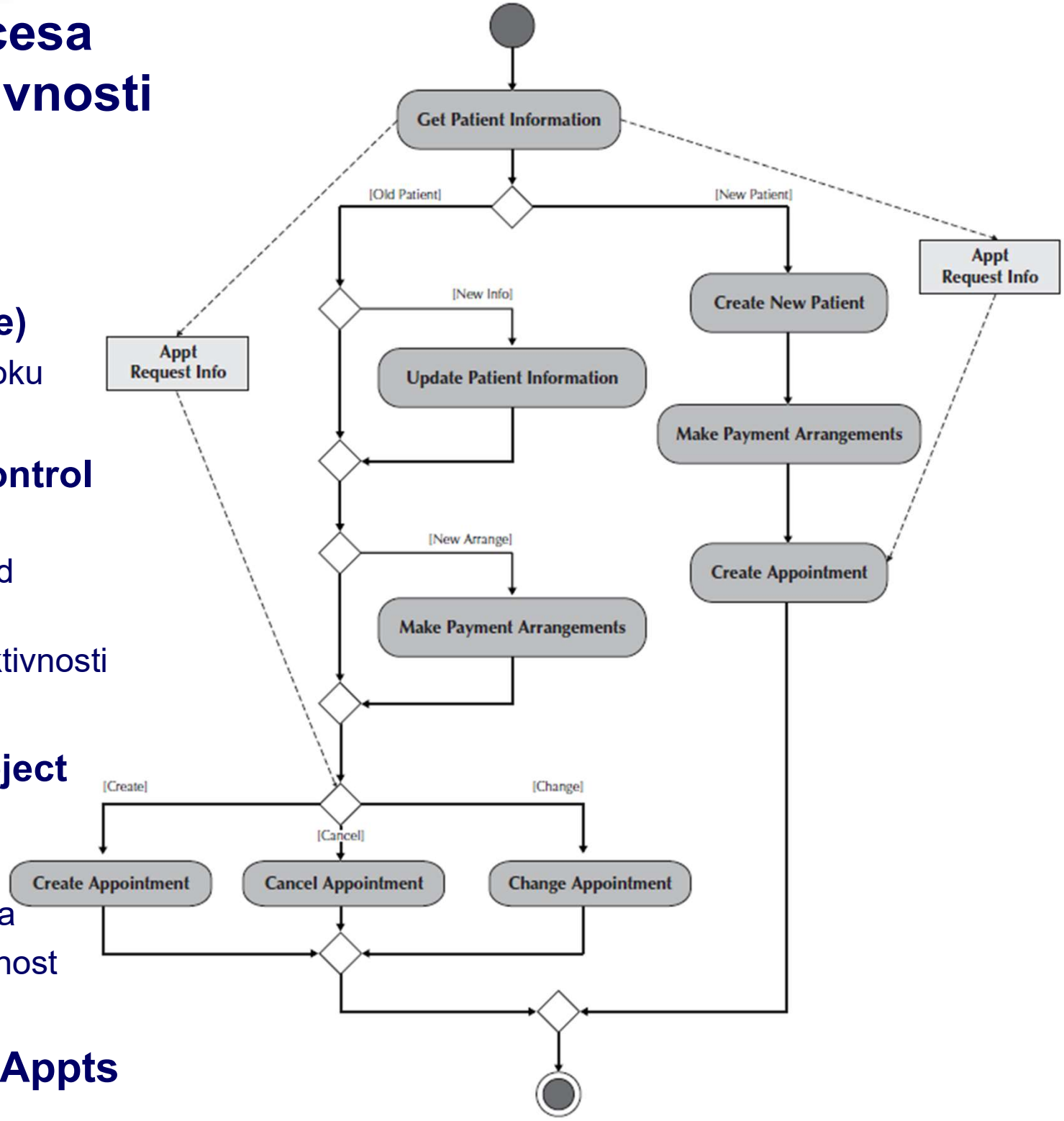
Primjer: UC dijagram

- Include
- Extend
- Generalizacija
- Opseg



Modeliranje procesa dijagramima aktivnosti

- **"sofisticirani DTP"**
  - dodatni elementi AD
- **Objekt (object node)**
  - klasa objekata u toku
- **Upravljački tok (control flow)**
  - puna strelica, slijed izvršenja
  - povezuje akcije/aktivnosti
- **Podatkovni tok (object flow)**
  - protok objekata
  - isprekidana strelica
  - veza objekt i aktivnost
- **Primjer: Manage Appts**



Slučajevi korištenja

- **Analiza slučajeva korištenja (Use Case Analysis)**
  - tehnika analize poslovnih procesa iz perspektive korisnika
- **Use case = Slučaj korištenja (primjene)**
  - **skup scenarija** sa zajedničkim ciljem korištenja sustava
- **Scenarij**
  - niz koraka koji opisuju interakciju korisnik-sustav
- **Tipični sadržaj**
  - kako kreće i završava slučaj
  - normalni tok događaja
  - varijabilni (alternativni) ili iznimni tok događaja
- **Standard izrade UC-a ne postoji (nije dio UMLa) !**
  - alternative i iznimke mogu biti zasebni slučajevi korištenja

cc BY NC SA

Primjer opisa slučaja korištenja 1/3

Naziv slučaja korištenja: Zakazati pregled	**ID:** 2	**Prioritet:** Visok
Glavni sudionik: Pacijent	**Tip slučaja korištenja:** Detaljni, osnovni	
Sudionici i interesi: Pacijent – želi zakazati, promijeniti ili otkazati pregled Liječnik – želi obraditi pacijenta u prihvatljivom vremenu		
Kratki opis: slučaj korištenja opisuje kako zakazati, promijeniti ili otkazati pregled.		
Okidač: Pacijent pozivom zakazuje novi pregled te mijenja ili otkazuje postojeći pregled. **Tip:** Vanjski		
Veze: Asocijacija (association): Pacijent Uključivanje (include): Izrada računa Proširenje (extend): Unos novog pacijenta Generalizacija (generalization):		
Tok događaja: 1. Pacijent kontaktira ordinaciju radi pregleda		

Elementi opisa slučaja korištenja

- Pregled (overview) - osnovni podaci o slučaju korištenja
  - naziv, tip, sudionici, kratak opis, prioritet
  - glavni, primarni sudionik – pokreće scenarij
  - okidač – događaj pokretanja (vanjski, unutarnji, vremenski ...)

- Veze - odnos s drugim slučajevima korištenja
  - Asocijacija (association) – veza između slučaja korištenja i sudionika
  - Proširenje (<<extend>>) – proširenje funkcionalnosti uz dodatna pravila
    - npr. *Unos novog pacijenta*, izvodi se uvjetno, nije dio normalnog toka
  - Uključivanje (<<include>>) – uključivanje drugog UC, prije <<uses>>
    - omogućuje funkcionalnu dekompoziciju (slično DTP)
    - npr. *Izrada računa* smatra se dovoljno složenom da bude samostalan UC
  - Generalizacija (generalization) – poopćenje i nasljeđivanje UC
    - npr. *Zakazivanje pregleda starom pacijentu* i *Zakazivanje pregleda novom pacijentu* imaju generalizirano ponašanje u *Zakazivanje pregleda*

BY NC SA

Primjer opisa slučaja korištenja 2/3

Uključivanje (include): Izrada računa
Proširenje (extend): Unos novog pacijenta
Generalizacija (generalization):

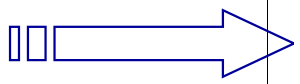
Tok događaja:

1. Pacijent kontaktira ordinaciju radi pregleda.
2. Pacijent službeniku daje svoje podatke.
3. Službenik potvrđuje da pacijent postoji u bazi pacijenata.
4. Službenik izvršava slučaj korištenja "Izrada računa".
5. Službenik pita pacijenta želi li zakazati novi pregled, promijeniti ili otkazati postojeći pregled.
   Ako pacijent želi zakazati novi pregled,
   izvodi se podtok T-1: novi pregled.
   Ako pacijent želi otkazati postojeći pregled,
   izvodi se podtok T-2: otkaži pregled.
   Ako pacijent želi promijeniti postojeći pregled,
   izvodi se podtok T-3: promijeni pregled.
6. Službenik obavještava pacijenta o rezultatu transakcije.

Podtokovi:

T-1: Novi pregled

1. Službenik pita pacijenta za moguće termine pregleda.
2. Službenik uspoređuje željene termine pacijenta i dostupne termine te zakazuje novi pregled.



Elementi opisa slučaja korištenja (nastavak)

- Tok događaja (flow of events) - opis koraka unutar procesa
  - normalni tok – standardni, uobičajeni koraci
  - podtokovi – zasebni slučajevi korištenja
    - definirani da bi se pojednostavnio normalan tok
    - slijede iz grananja dijagrama aktivnosti
    - mogu se oblikovati uključivanjem u normalan tok (<<include>>)
  - alternativni/izuzetni tokovi – mogu se dogoditi, ali nije uobičajeno
    - npr. potreba za unosom novog pacijenta prije nego mu se zakaže pregled

- **Navedeni su minimalni elementi, "zaboravljeno" je:**
  - procjena vremena potrebnog za realizaciju
  - specifični tokovi podataka
  - atributi, ograničenja ili operacije u vezi sa slučajem korištenja
  - pretpostavke (pre-conditions) i posljedice (post-conditions)
  - ...

cc BY NC SA

Primjer opisa slučaja korištenja 3/3

❑ Primjer: 🗁Dizajn\UCprimjer

izvodi se podtok T-3: promijeni pregled.

6. Službenik obavještava pacijenta o rezultatu transakcije.

Podtokovi:

T-1: Novi pregled

1. Službenik pita pacijenta za moguće termine pregleda.
2. Službenik uspoređuje željene termine pacijenta i dostupne termine te zakazuje novi pregled.

T-2: Otkaži pregled

1. Službenik pita pacijenta za termin zakazanog pregleda.
2. Službenik pronalazi zakazani pregled i otkazuje ga.

T-3: Promijeni pregled

1. Službenik izvodi podtok T-2: otkaži pregled.
2. Službenik izvodi podtok T-1: novi pregled.

Alternativni/izuzetni tokovi:

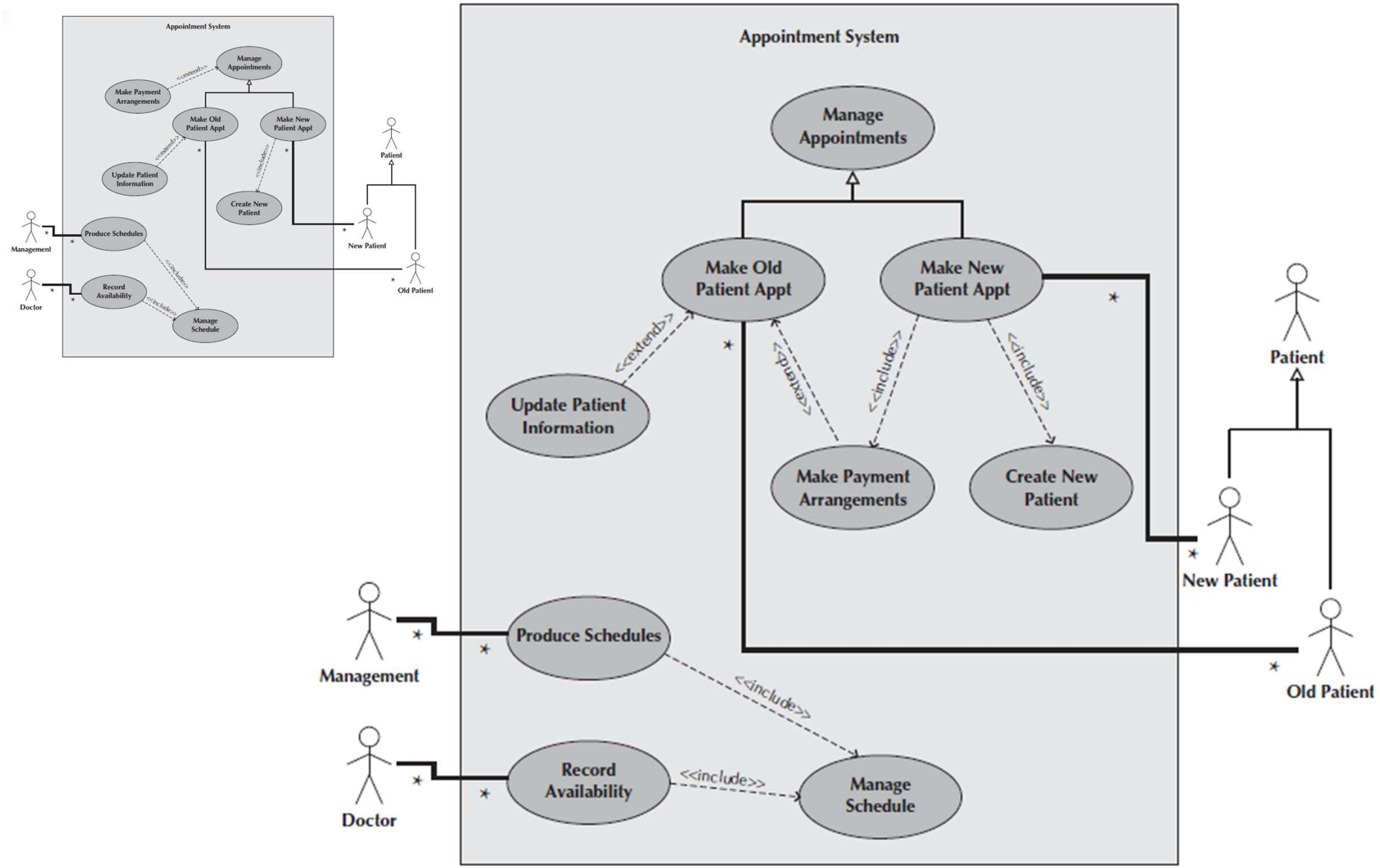
3a: Službenik izvodi slučaj korištenja "Unos novog pacijenta".

T-1, 2a1: Službenik u skladu s dostupnim terminima predlaže alternativne termine pregleda.

T-1, 2a2: Pacijent se odlučuje za jedan od predloženih termina ili odlučuje ne zakazati pregled.

BY NC SA

Primjer: UC dijagram revidirani



Dijagrami aktivnosti naspram slučajeva korištenja

❑ Slučaj korištenja

- opisuje samo jednu funkciju interakcije korisnika i sustava
- može uključivati različite scenarije
  - Npr. scenariji za isti UC: pretraživanje knjiga po autoru, po naslovu, ...
- UC smije biti povezan s jednom i samo jednom ulogom
  - Npr. liječnička "sestra" dogovara preglede, ispisuje uputnice i recepte, naplaćuje participaciju ...

❑ Dijagram aktivnosti

- analiza jednog slučaja korištenja za proces povezan s jednom ulogom
- slijed akcija za nekoliko objekata i nekoliko UC (posljedično više uloga)
- razumijevanje radnog, poslovnog procesa
- opisivanje složenih slijednih algoritama (obrade podataka)
- opisivanje višenitnih aplikacija

BY NC SA

Komentar dijagrama i daljnji koraci

- **Granica sustava (subject boundary) – doseg projekta**
  - razdvaja vanjske i unutarnje dijelove (pravokutnik oko slučajeva)
  - na prethodnoj slici nedostaje *Službenik*, jer je dio sustava (sudionik)

- **Mogući daljnji koraci**
  - Rafiniranje veličine projekta UC točkama - slično funkcijskim točkama
  - Modeliranje struktura - u nastavku

- **Primjeri: 🗁Dizajn\UC***

Modeliranje struktura

Structural modeling

BY NC SA

Modeliranje struktura

- **Model strukture (structural model)**
  - formalna prezentacija objekata kreiranih i korištenih u poslovnom sustavu
  - opisuje strukturu podataka koji prate poslovni proces

- **Modeliranje strukture**
  - modelira se iterativno od konceptualne do detaljne razine
  - u fazi analize bez ulaženja u detalje pohrane podataka

- **Primjenjive tehnike**
  - CRC kartice
  - dijagrami razreda
  - dijagrami objekata

cc BY NC SA

CRC kartice

- **Razred-Odgovornost-Suradnja (*Class-Responsibility-Collaboration*)**
- **CRC kartica**
  - modelira odgovornosti jednog razreda i njegovu suradnju s drugim razredima
  - može se koristiti umjesto dijagrama suradnje
  - preporuča se da sadrži najviše tri odgovornosti visoke razine
    - u protivnom – razmisliti o razdvajanju razreda u nekoliko manjih
- **Odgovornosti**
  - odgovornost znanja (podaci)
  - odgovornost radnje (operacije, postupci)
- **Suradnja**
  - klijent (client) – instanca koja šalje zahtjev drugoj instanci za izvođenjem
  - poslužitelj (server) – instanca koja prima zahtjev
  - ugovor (contract) – formalizacija interakcije između klijenta i poslužitelja
    - npr. pacijent dogovara pregled s liječnikom
    - oba sudionika imaju obvezu dolaska na pregled
    - ukoliko se pacijent ne pojavi, otpada plaćanje računa

cc BY NC SA

Primjer CRC kartice

Prednja stranica:

Razred: Pacijent	**ID:** 3	**Tip:** Konkretni, Domenski
Opis: Pojedinac koji treba ili prima medicinsku pomoć.		**Slučajevi korištenja:** 2

Odgovornosti	**Suradnici**
Zakazati pregled_____	Pregled_____
Odrediti zadnji posjet_____	_____
Promijeniti status_____	_____
Dohvatiti povijest bolesti_____	Povijest bolesti_____
_____	_____
_____	_____

Stražnja stranica:

Atributi:

Iznos (double)_____
Nositelj osiguranja (text)_____
_____
_____

_____
_____
_____
_____

Veze:

Generalizacija (a-kind-of): Osoba_____
Agregacija (has-parts): Povijest bolesti_____
Ostale veze : Pregled_____

Koraci modeliranja strukture

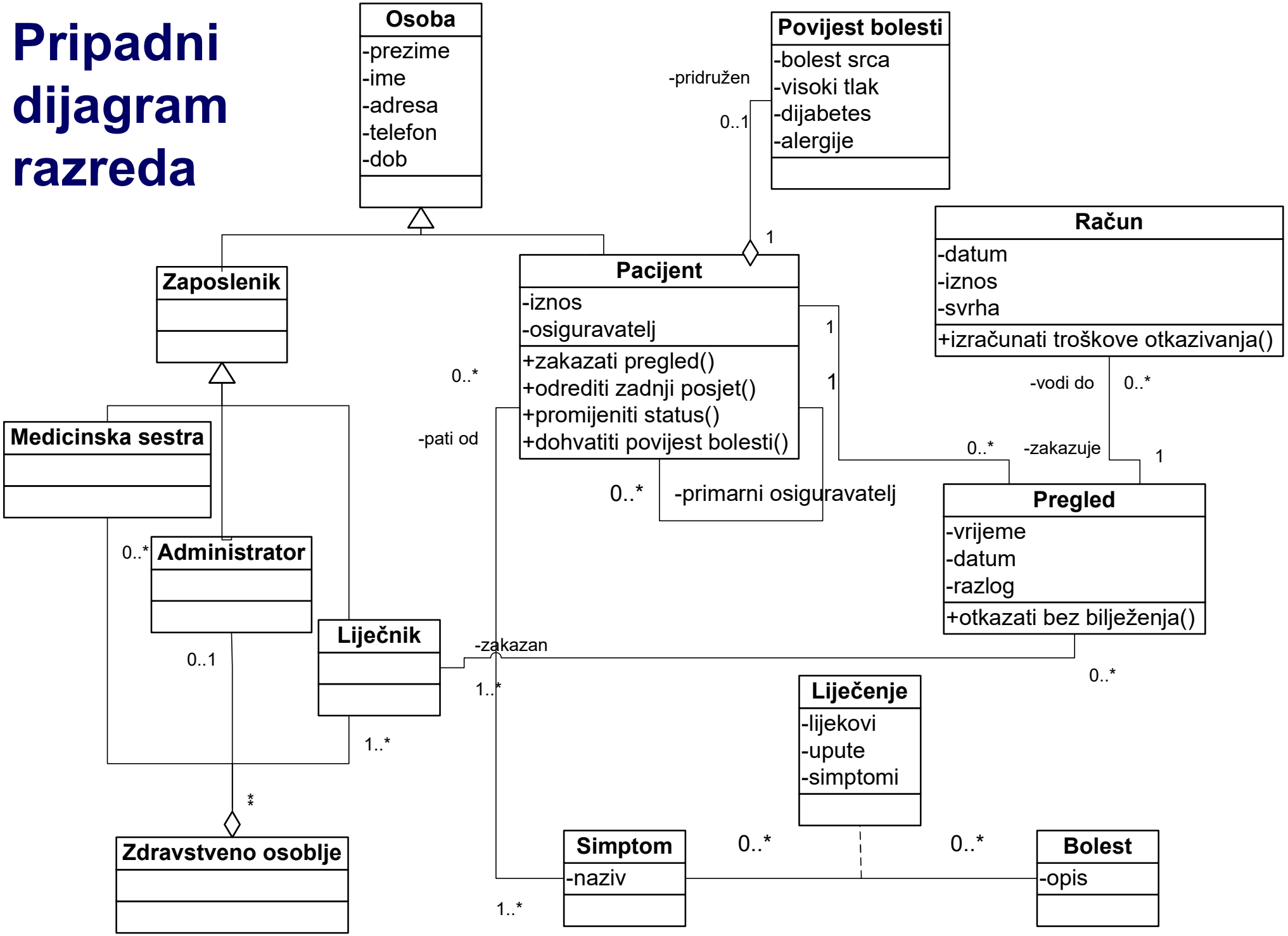
- **Izrada CRC kartice analizom opisa UC**
  - određivanje razreda, atributa, operacija i asocijacija

- **Analiza skupne liste objekata (common object list approach)**
  - opipljive stvari (tanglible things) – npr. oprema, materijal, roba
  - incidenti (incidents) – događaji, npr. sastanci, letovi, akcidenti
  - uloge (roles) – uloge osoba, npr. liječnici, pacijenti, administratori
  - interakcija (interaction) – transakcije, npr. akcija prodaje

- **Glumljenje CRC karticama (role-play the CRC cards)**
  - testiranje UC – antropomorfizam – igrač uzme karticu i glumi razred
    - Tko si ili što si ? Što znaš ? Što možeš ?

- **Primjena obrazaca (incorporate patterns)**
  - "kako se to radi"
    - analitički obrasci, npr. *Partner-Dokument-Stavke-Artikl*
  - “kako realizirati”, “kako ponovno iskoristiti”
    - oblikovni obrasci, aplikacijski okviri, knjižnice razreda

Pripadni dijagram razreda



Dizajn vođen odgovornostima

cc BY NC SA

Dizajn vođen odgovornostima

❑ Responsibility-Driven Design (RDD)

- postupak oblikovanja softvera koji naglašava modeliranje uloga i odgovornosti objekata te suradnje objekata

❑ Načela dizajna vođenog odgovornostima

- Maksimiziranje apstrakcije
  - početno se skriva (zanemaruje) razlika između podataka i ponašanja
  - odgovornosti objekta su "znanje", "činjenje", "odlučivanje"
- Distribucija ponašanja
  - poboljšavanje odabrane upravljačke arhitekture
  - pametni objekti – inteligentno ponašanje, a ne samo pohrana podataka
- Očuvanje elastičnosti, fleksibilnosti
  - projektiranje objekata (razreda) koje se može brzo i lako mijenjati

BY NC SA

Pristup oblikovanju vođenom odgovornostima

- **Postupak oblikovanja vođenog odgovornostima**
  - Analiza slučajeva korištenja ili korisničkih priča
  - Izrada liste potencijalnih poslovnih objekata i njihovih veza
  - Određivanje (jedinstvene) odgovornosti objekata
  - Postupnim pročišćavanjem nastane dizajn razreda
  - Temeljem istih slučajeva korištenja projektira se (relacijski) model podataka

  .... (kodiranje, refaktoriranje)

- **Dizajn jednostruke odgovornosti**
  - Objekti postoje da bi podržali UC/scenarij
  - Pojedini objekt treba imati jednu jednostruku odgovornost unutar UC
  - Objekt može imati više ponašanja (svojstava, postupaka), ali takvih da objekt podupire jedinstvenu odgovornost
  - Pažnja se usmjerava na to što objekt radi a ne na podatke !

BY NC SA

Restrukturiranje i optimiranje dizajna

❑ Restrukturiranje dizajna

- Faktoriranje
  - postupak izdvajanja novog modula (razreda, metode) iz postojećih
  - npr. da u primjeru ordinacije nije bio identificiran *Zaposlenik*, izvjesno bismo ga izlučili iz *Sestra*, *Administrator* ili *Doktor*
- Normalizacija
  - ustanovljavanje razreda koji nedostaju
  - pretvorba asocijacijskih razreda i agregacija u atribute

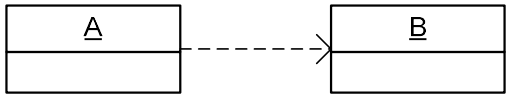
❑ Optimiranje dizajna (refaktoriranje - u fazi kodiranja)

- skraćenje pristupnih putova između objekata
- smanjenje broja metodom poslanih poruka
- redoslijed izvršenja naredbi (npr. redoslijed pretraživanja)
- uvođenje izvedenih atributa umjesto ponavljanog izračuna

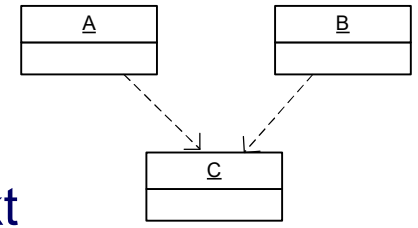
BY NC SA

Zavisnost objekata i normalizacija ponašanja

- **Npr. Objekt A i Objekt B podupiru svaki svoj UC**
  - A treba neko ponašanje od B (kopčanje, eng. coupling)
  - svaki od objekata podržava vlastiti UC
  - rizik da će se B promijeniti radi promjene zahtjeva na UC koji podržava



- **Normalizacija ponašanja**
  - postupak postizanja ponovne iskoristivosti i kontrole kopčanja
  - izdvajanje koda koji se želi ponovno iskoristiti u zaseban objekt
  - npr, ponašanje iz B izdvaja se u C, te A i B nastavljaju nezavisno kolaborirati



- **Ponovnu iskoristivost forsirati samo ako se može izbjeći kopčanje !**

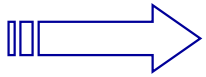
Primjer dizajna vođenog odgovornostima

Modeliranje analizom više slučajeva korištenja

CC BY NC SA

Primjer RDD: Slučajevi sustava evidencije projekata

- **Dodavanje projekta (provodi upravitelj projekta)**
  - podaci o projektu sadrže naziv, opis, datum početka i datum završetka
  - projekt se identificira nazivom, pa je to zahtijevana vrijednost
  - ostale vrijednosti su opcionalne
  - moguće je dodati projekte koji još nisu započeti
  - započeti projekt ne mora imati datum završetka
  - datum početka mora biti manji od datuma završetka
  - svaki projekt ima listu sudionika (pogledati Pridjeljivanje sudionika)



- **Ažuriranje projekta (provodi upravitelj projekta)**
  - moguće je ažurirati postojeći projekt odabirom s liste projekata
  - može se ažurirati datum početka, datum završetka i opis
  - moguće je ažurirati listu sudionika (pogledati Pridjeljivanje sudionika)

- **Brisanje projekta (upravitelj projekta ili administrator)**
  - brisanje se obavlja odabirom projekta s liste projekata

cc BY NC SA

Evidencija sudionika

- **Dodavanje osoba (upravitelj projekta ili rukovoditelj)**
  - pohranjuju se ime i identifikator zaposlenika, uz obvezan unos
  - treba omogućiti evidenciju projekata osobe (Pridjeljivanje sudionika)
- **Ažuriranje osobe (upravitelj projekta ili rukovoditelj)**
  - dozvoliti izmjenu imena
- **Brisanje osobe (upravitelj projekta ili rukovoditelj)**
  - zaposlenik može prekinuti posao ili otići u drugi dio organizacije
  - brisanje se obavlja odabirom osobe s liste osoba

- **Pridjeljivanje sudionika (upravitelj projekta)**
  - proces se pojavljuje u više drugih procesa, pa je izdvojen u zaseban slučaj
  - osoba se dodjeljuje projektu, odabirom uloge iz liste uz automatski datum
  - treba dozvoliti promjenu uloge osobe
  - osoba može sudjelovati u više projekata
  - treba omogućiti brisanje zaduženja

- **Evidencija uloga (administrator)**
  - omogućiti održavanje uloga u odgovarajućem šifrarniku

cc BY NC SA

Projektiranje objekata (Object Design)

- **kombiniranjem dekompozicije ("traženjem imenica") i CRC karticama**

- **Početni dizajn - ustanovljavanje potencijalnih entiteta**
  - neki od nabrojanih entiteta nisu objekti nego npr. uloge korisnika sustava (Upravitelj projekta, Administrator, Rukovoditelj), a neki su atributi

Upravitelj Naziv projekta Administrator Osoba Rukovoditelj Lista uloga Lista osoba	Projekt Datum početka Lista projekata Naziv zaposlenika Lista projekata osobe Zaduženje Lista sudionika projekta	Broj projekta Datum završetka Zaposlenik Id. zaposlenika Uloga Datum zaduženja

- **Lista entiteta se reducira te se dodaju odgovornosti i suradnici**
- **Napomena: dalje se nećemo baviti aspektima sigurnosti i zaštite**

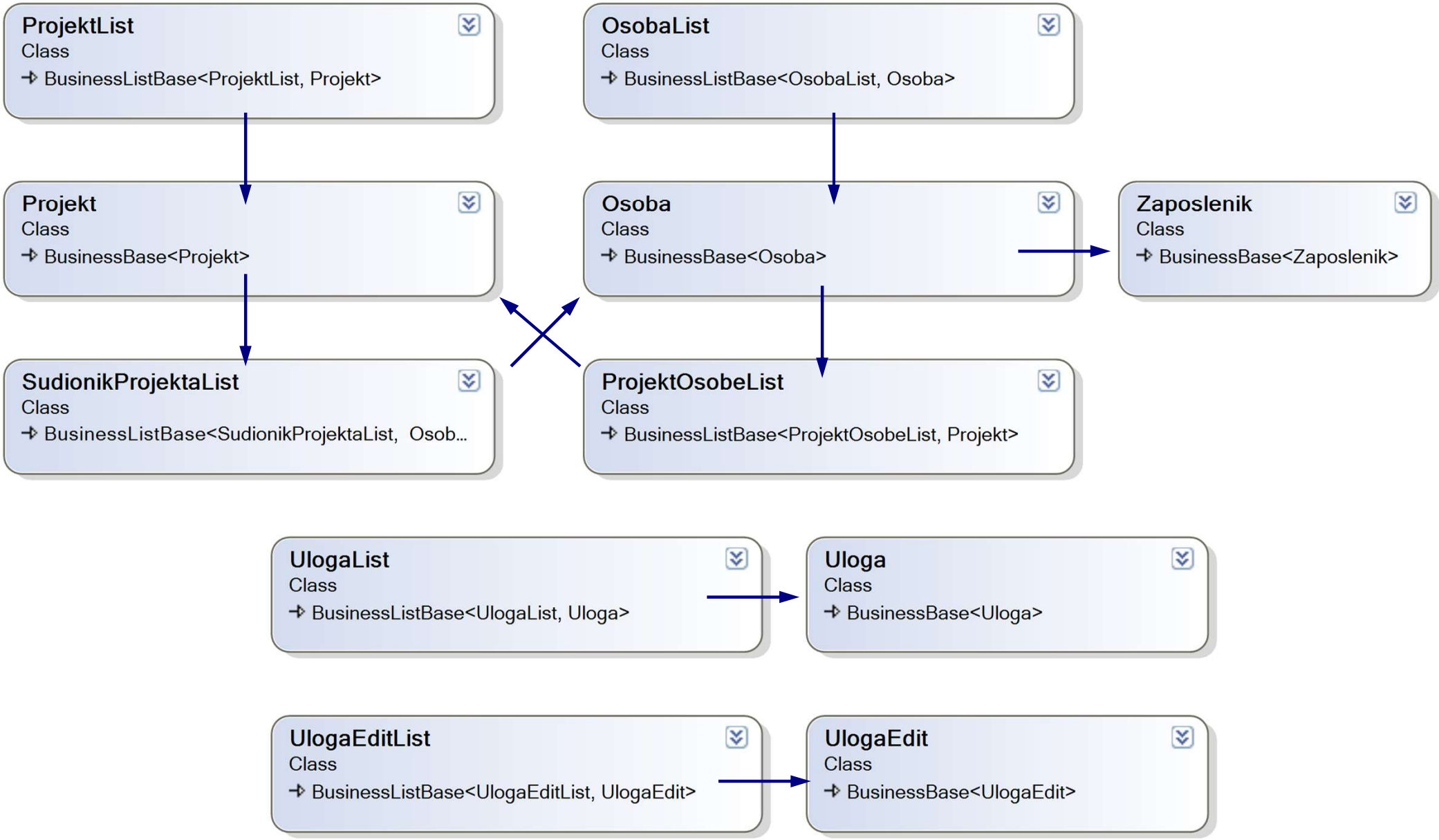
BY NC SA

Razredi potencijalnih objekata

- **CRC kartica visoke razine za potencijalne objekte:**
  - odgovornost treba biti izražena kratko i precizno
  - ako nije, to može značiti da je riječ o složenom objektu koji treba rastaviti

Potencijalni razred	Odgovornost	Suradnici
`Projekt`	dodaje i ažurira ispravan projekt	`SudionikProjektaList`
`Osoba`	dodaje i ažurira ispravnu osobu	`ProjektOsobeList, Zaposlenik`
`Zaposlenik`	dodaje i ažurira ispravnog zaposlenika	
`ProjektList`	dobavlja read-only listu projekata	`Projekt`
`OsobaList`	dobavlja read-only listu osoba	`Osoba`
`SudionikProjektaList`	održava listu osoba dodijeljenih projektu	`Osoba, UlogaList`
`ProjektOsobeList`	održava listu projekata na kojima je osoba	`Projekt, UlogaList`
`UlogaList`	dobavlja read-only listu uloga	`Uloga`
`Uloga`	read-only vrsta uloge	
`UlogaEditList`	održava listu uloga	`UlogaEdit`
`UlogaEdit`	dodaje i ažurira ispravnu ulogu	

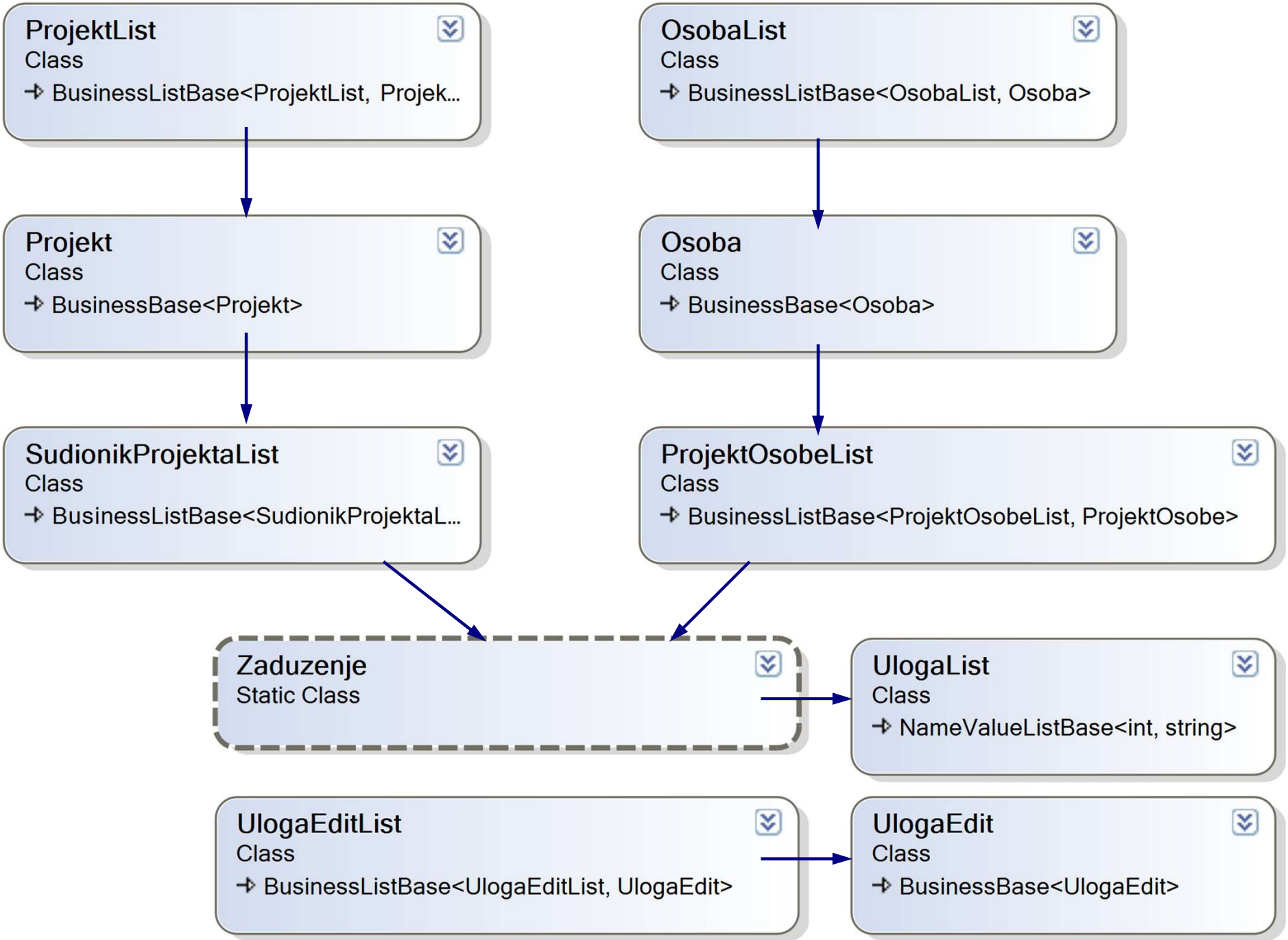
Dijagram potencijalnih razreda



Revizija dizajna

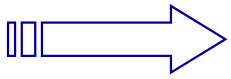
- **`Osoba` i `Zaposlenik` su potencijalni duplikati**
  - nema naznaka da osoba dodaje bilo što zaposleniku
  - `Osoba` i `Zaposlenik` mogu se udružiti
- **`UlogaList` i `Uloga`**
  - `UlogaList` je jednostavna lista oblika šifra/naziv (tzv. name/value list)
  - `Uloga` je jednostavan šifrarnički zapis (tzv. name/value placeholder)
  - može se kasnije pojednostavniti odgovarajućim razredom (npr. `Csla.NameValueListBase`)
- **`Projekt, SudionikProjektaList, Osoba, ProjektOsobeList`**
  - komplicirane, cirkularne veze – potencijalne beskonačne petlje
  - nedostaje podatak o ulozi osobe na projektu
    - osoba treba sudjelovati na više projekata ili u više uloga
    - uloga za sada nepovezana
  - rješenje: dodavanje novog razreda `Zaduzenje`

Revidirani dijagram



Problem zaduženja

- **Razred `Zaduzenje` koristi se u dva konteksta**
  - s liste sudionika projekata i s liste projekata osobe
  - zajedničko dijete različitih kolekcija komplicira ugradnju i testiranje
- **Odgovornost razreda `Zaduzenje` ? - nejednoznačno**
  - pridruživanje osobe projektu
  - pridruživanje projekta osobi
- **Problem s podacima**
  - `Projekt` koristi `SudionikProjektaList` da dohvati listu osoba projekta
    - podrazumijeva da `Zaduzenje` sadrži informaciju o osobi
  - `Osoba` koristi `ProjektOsobeList` za dohvat liste projekata osobe
    - podrazumijeva da `Zaduzenje` sadrži informaciju o projektu
- **Očigledno postoji konflikt ponašanja ali i konflikt podataka**
- **Moguća rješenja**
  - udruživanje `SudionikProjektaList` i `ProjektOsobeList` u npr. `ZaduzenjeList`
    - problem bi se izgledno preselio razinu više (pa se nećemo time baviti)
  - alternativa: dijeljenje razreda `Zaduzenje` u dva zasebna razreda,
    - `SudionikProjekta` i `ProjektOsobe`

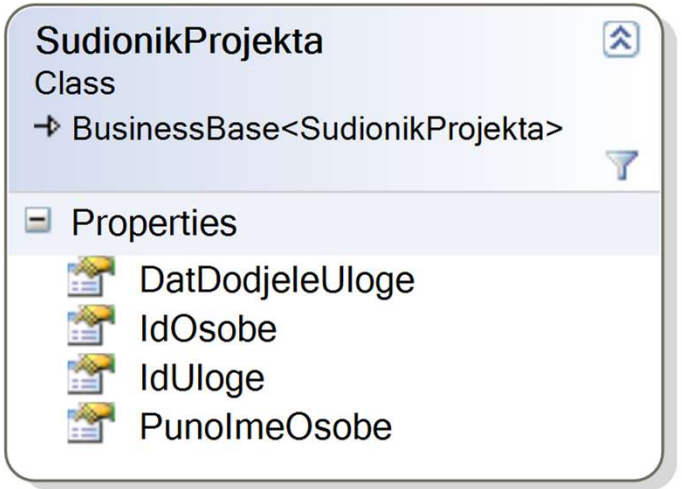


Alternativa (1.dio) : Dodjela osobe projektu

- **Temeljem UC, korisnik odabere projekt i dodaje osobu projektu**
  - podrazumijeva da projekt ima kolekciju pridijeljenih osoba

- **Koja ponašanja (odgovornosti) očekivati od `SudionikProjektaList` ?**
  - vratiti listu osoba pridruženih projektu
  - pridruživanje nove osobe projektu, npr. postupkom `Dodijeli(IdOsobe)`
  - davanje podataka za pregled i uređivanje

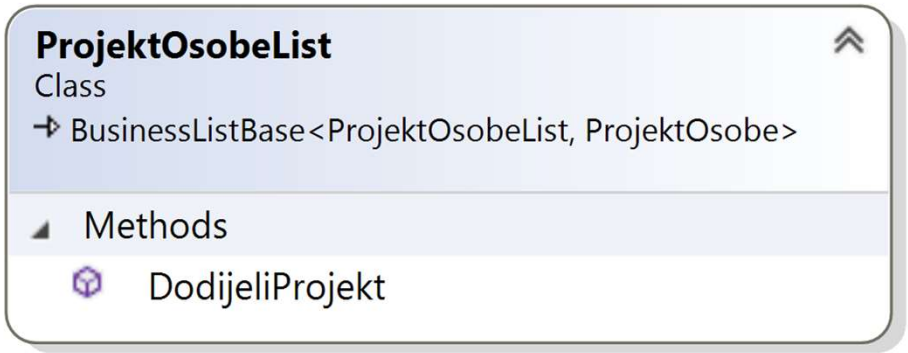


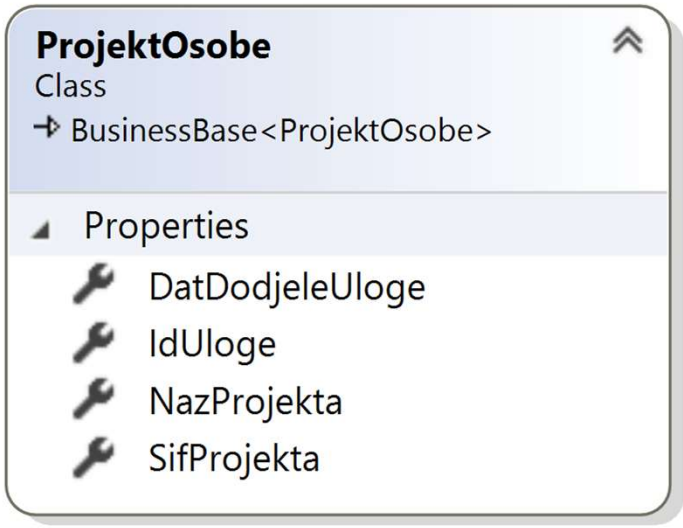


Alternativa (2.dio) : Dodjela projekta osobi

- **Analogno prethodnoj dodjeli, dodaje se projekt odabranoj osobi**
  - podrazumijeva da osoba ima kolekciju pridijeljenih projekata

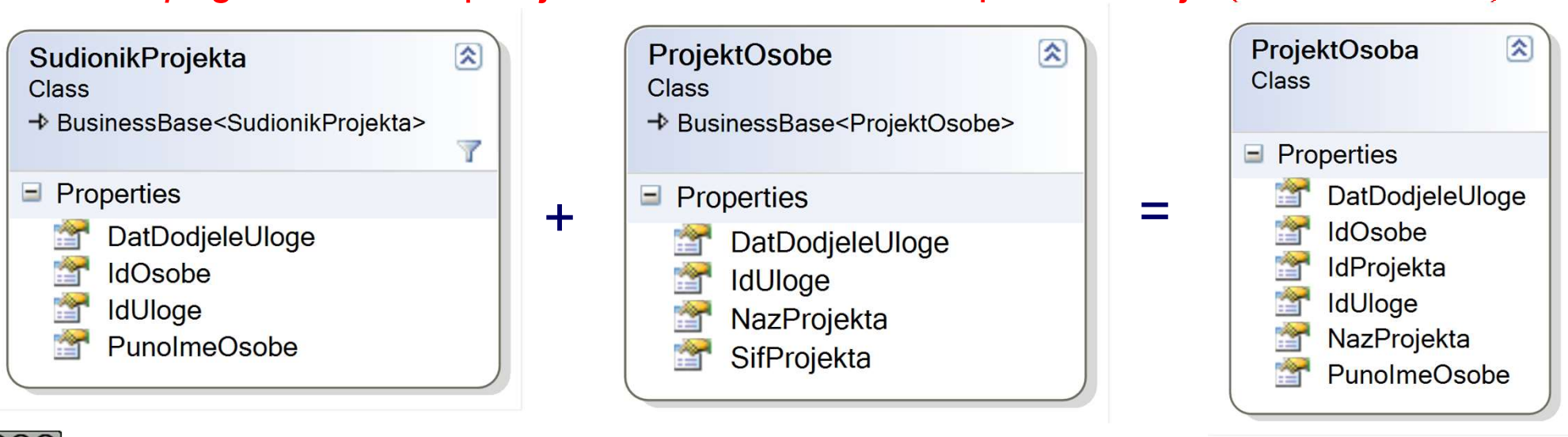
- **Koja ponašanja (odgovornosti) očekivati od `ProjektOsobeList` ?**
  - vratiti listu projekata pridruženih osobi
  - pridruživanje projekta osobi, npr. postupkom `DodijeliProjekt(SifProjekta)`
  - davanje podataka za pregled i uređivanje





Što dalje: Udruživanje sličnih razreda ?

- **`SudionikProjekta` i `ProjektOsobe` sadrže slične podatke, ali im je ponašanje različito**
- **Smislenost udruživanja u zajednički razred `ProjektOsoba` ?**
  - Naziv projekta nema smisla kada se objekt referencira iz konteksta projekta
  - slično vrijedi za neka druga svojstva, ovisno o kontekstu
- **Rješenje?**
  - poslovna logika koja će baciti iznimku ako se objektu `ProjektOsoba`, ovisno o kontekstu, pristupa na pogrešan način
  - ostaje, međutim, problem da `ProjektOsoba` ima različite odgovornosti
  - *Spaghetti code* – povijesno dokazano loša implementacija (dakle ne tako)



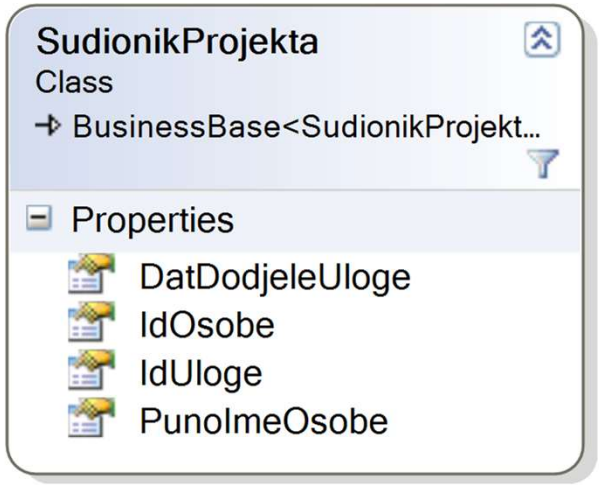
Vladanje zajedničkim ponašanjem i informacijama

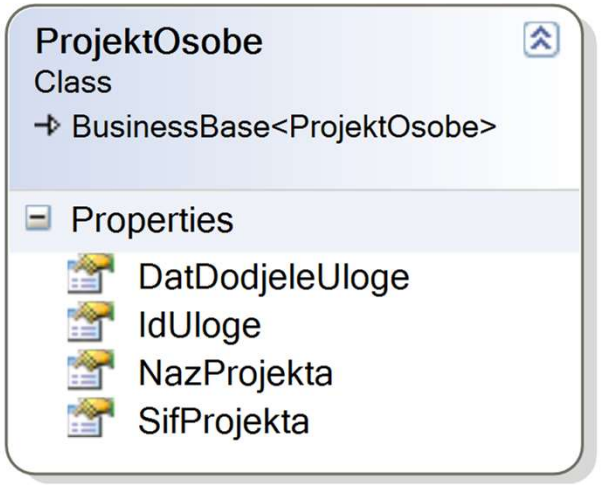
- **Kako koristiti iste podatke bez umnažanja poslovne logike ?**
  - normalizacija ponašanja
- **Osnovna ideja relacijskih baza podataka je smanjenje zalihosti**
  - svaki elementarni podatak smije postojati samo jednom
- **Cilj objektnog dizajna je osiguranje jedinstvenog ponašanja**
  - svako ponašanje mora postojati samo jednom u modelu, ali smije biti korišteno od strane različitih objekata

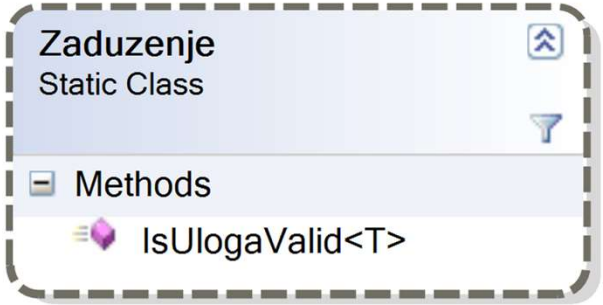
cc BY NC SA

Vladanje zajedničkim informacijama

- **`DatDodjeleUloge` i `IdUloge` ne treba smještati u zajednički objekt**
  - jer ne razmišljamo relacijski, ne normaliziramo podatke
- **Pitanje je imaju li ti atributi zajedničko ponašanje (poslovno pravilo ili poslovnu logiku) koje se može staviti u zajednički objekt**
  - Primjer: validacija dodijeljene uloge

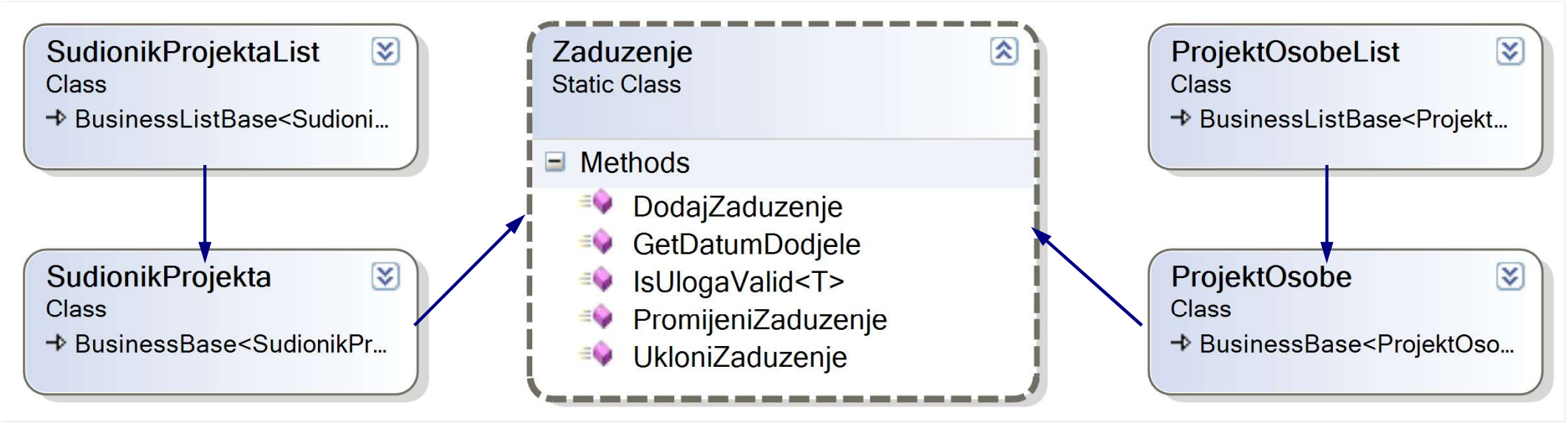






Vladanje zajedničkim ponašanjima

- **Odgovornost razreda `Zaduzenje` je upravljanje vezom između projekta i osobe kao resursa projekta**
  - `Zaduzenje` treba podržati pridruživanja osobe projektu i obrnuto
  - Posao dodjele vodit će `SudionikProjekta` i `ProjektOsobe`
    - Koji znaju šifru projekta, odnosno osobe
  - kolekcijski razredi samo će dodavati i uklanjati ove objekte
  - naravno, koristiti i druge postupke razreda `Zaduzenje`



Poboljšanje performanci

- **`SudionikProjektaList` i `ProjektOsobeList` dohvaćaju kolekcije poslovnih objekata `Projekt` i `Osoba` samo da bi ih prikazali u listi**
  - učitavanje objekta `Projekt` ili `Osoba` povlači učitavanje lista zaduženja ... (kaskadno)
- **Umjesto učitavanja hijerarhija - formirati read-only liste**
  - Uvode se potrebni read-only objekti za prikaz osoba i projekata `ProjektInfoList`, `ProjektInfo`, ...



Ponovna procjena dizajna

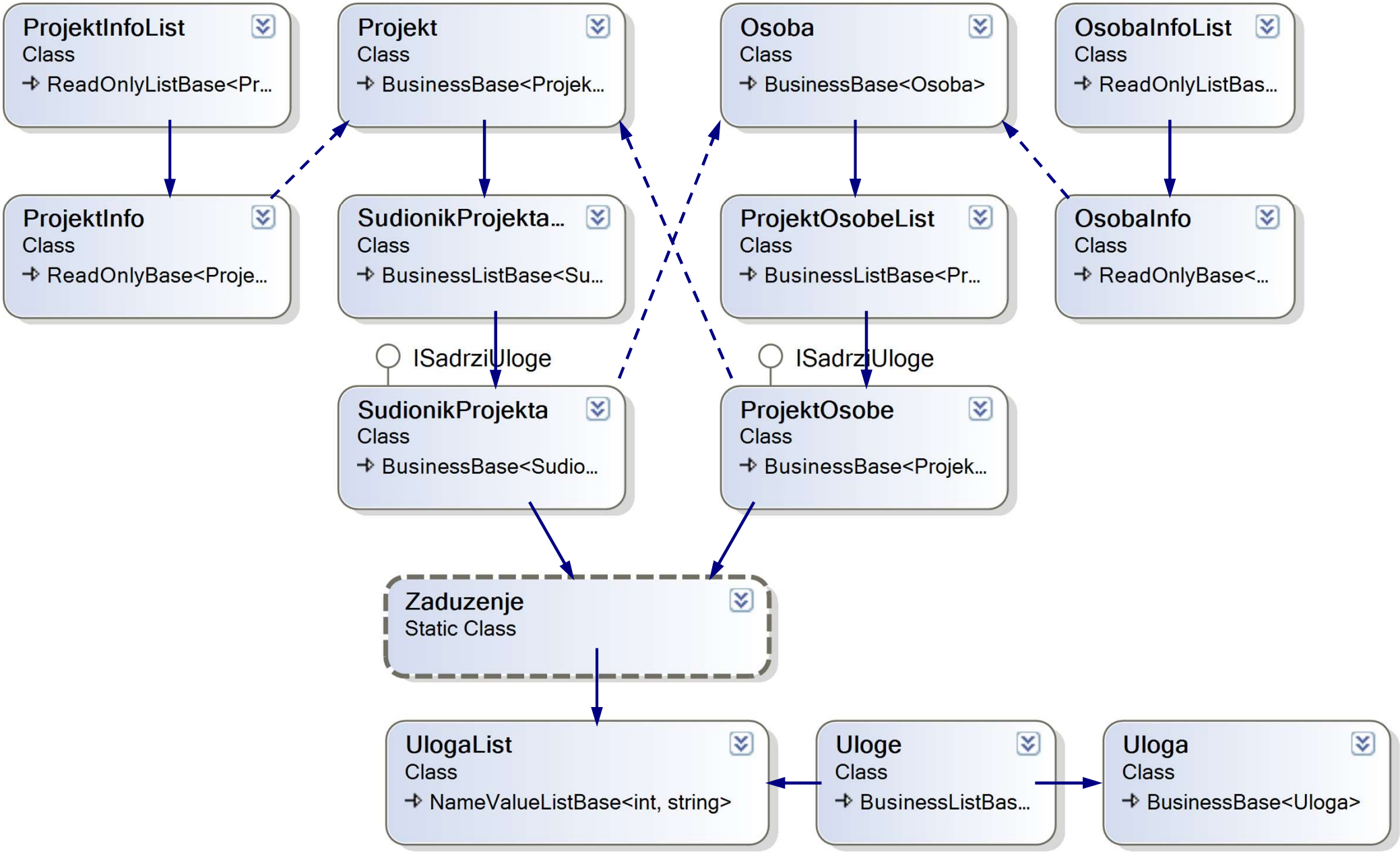
- **Završni korak - usporedba konačnog modela s izvornim UC**

- **U nastavku je prikazan dijagram koji**
  - ilustrira suradnju objekata (pune poveznice)
  - pokazuje navigaciju između objekata (isprekidane poveznice)
    - npr. s `ProjektInfo` objekta moguća je navigacija prema objektu Projekt, postupkom `GetProjekt()`
  - primijetiti da je `ProjektList` zamijenjen s `ProjektInfoList`, a `OsobaList` s `OsobaInfoList`

- **Iza slijedi konačna lista objekata i njihovih odgovornosti**

cc BY NC SA

Konačni objektni model evidencije projekata



Konačna lista objekata i odgovornosti

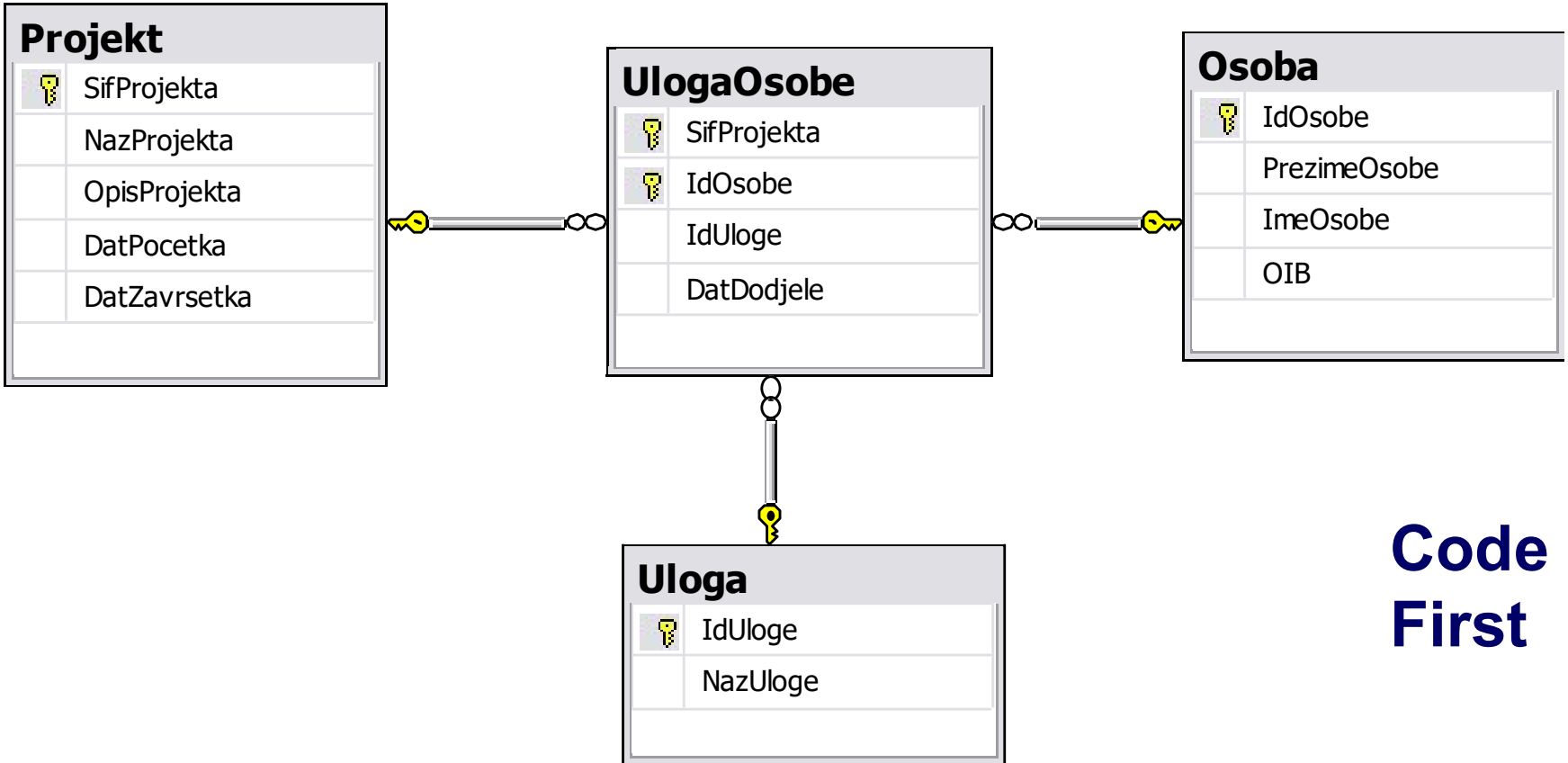
Razred	Odgovornost	Suradnici
`Projekt`	dodaje i ažurira ispravan projekt	`SudionikProjektaList`
`SudionikProjektaList`	održava listu osoba dodijeljenih projektu	`SudionikProjekta`
`SudionikProjekta`	upravlja pridjeljivanjem osobe projektu	`Zaduzenje, Osoba`
`Osoba`	dodaje i ažurira ispravnu osobu	`ProjektOsobeList, Zaposlenik`
`ProjektOsobeList`	održava listu projekata na kojima je osoba	`ZaduzenjeOsobe`
`ZaduzenjeOsobe`	upravlja pridjeljivanjem projekta osobi	`Zaduzenje, Projekt`
`Zaduzenje`	održava vezu projekta i osobe	`UlogaList`
`ProjektInfoList`	dobavlja read-only listu projekata	`ProjektInfo`
`ProjektInfo`	read-only informacija o projektu	`Projekt`
`OsobaInfoList`	dobavlja read-only listu osoba	`OsobaInfo`
`OsobaInfo`	read-only informacija o osobi	`Osoba`
`UlogaList`	dobavlja read-only listu uloga	
`Uloge`	održava listu uloga	`Uloga, UlogaList`
`Uloga`	dodaje i ažurira ispravnu ulogu	

cc BY NC SA

Na kraju

- **Relacijski dizajn i OO dizajn**
  - 2 različita procesa koji rezultiraju različitim modelima
  - Usklađivanje ovih modela provodi se objektno-relacijskim mapiranjem

- **Object Relational Mapping (ORM, OR/M)**
  - tehnika konverzije podataka između relacijske BP i OO jezika



Code First



CC BY NC SA

Prilozi i reference

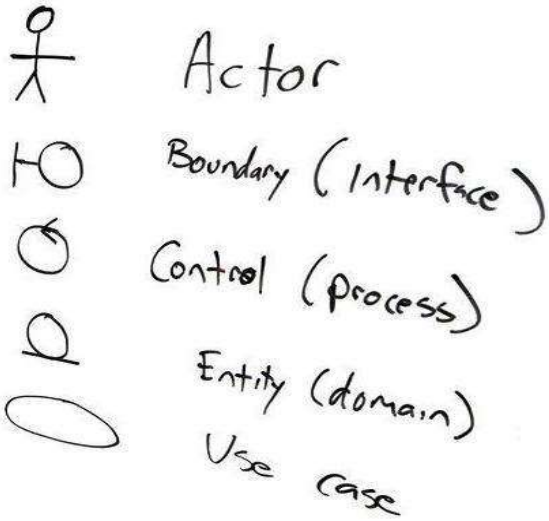
🗁Primjeri 🗁IS06-dodatak

❑ Reference

- Dennis et al. System Analysis and Design with UML, Wiley, 2015.
- Rockford Lhotka. Expert C# 2008 Business Objects, Apress, 2008.
- Rebecca Wirfs-Brock: A Brief Tour of Responsibility-Driven Design
- Vaughn Vernon. Domain-Driven Design Distilled, Addison-Wesley Professional, 2016.
- Avram i Marinescu. Domain-Driven Design Quickly, InfoQ, 2007.

❑ Agilno modeliranje

- http://www.agilemodeling.com
- .../artifacts/robustnessDiagram.htm



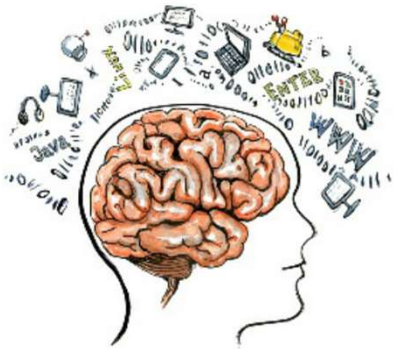
Domenom vođen dizajn

CC BY NC SA

Domain-Driven Design (DDD)

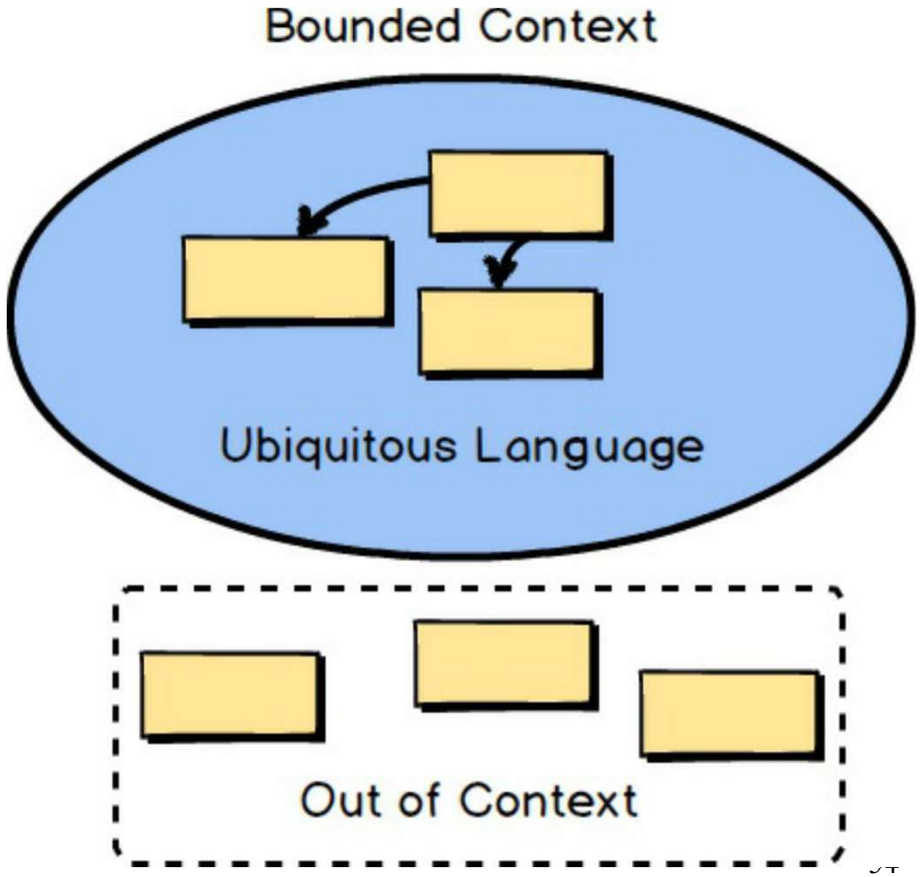
- Domena – problemsko područje
- DDD – projektiranje softvera na temelju ulaza stručnjaka domene
- *Sveprisutni jezik* – razumljiv razvojnicima i *domencima*
- *Ograničeni kontekst* – granica/opseg (pod)domene

Developers



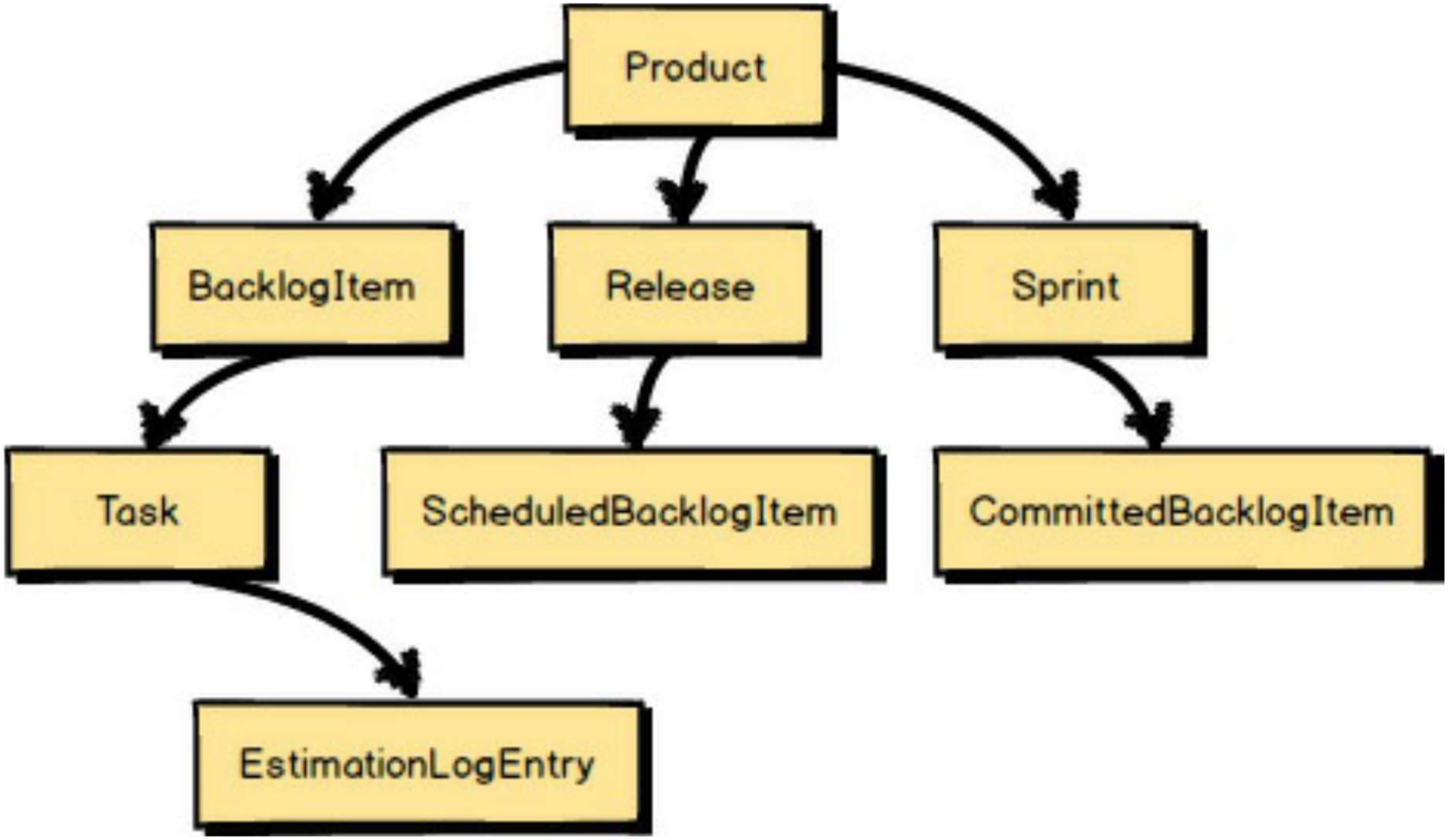
Domain Experts





Primjer, zahtjevi na programsko rješenje

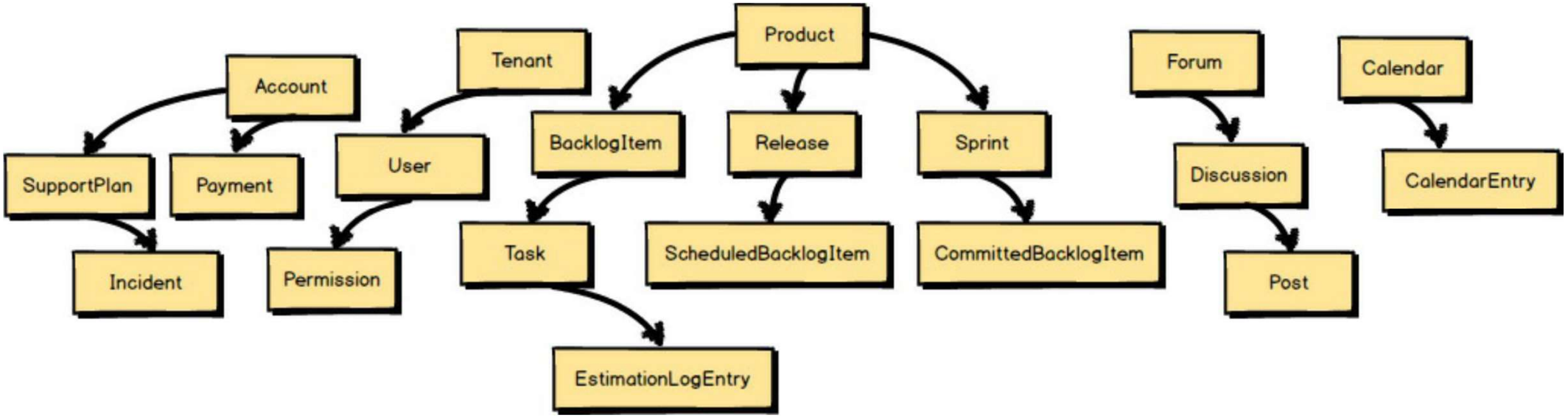
- Platforma za agilno upravljanje projektom metodom Scrum



BY NC SA

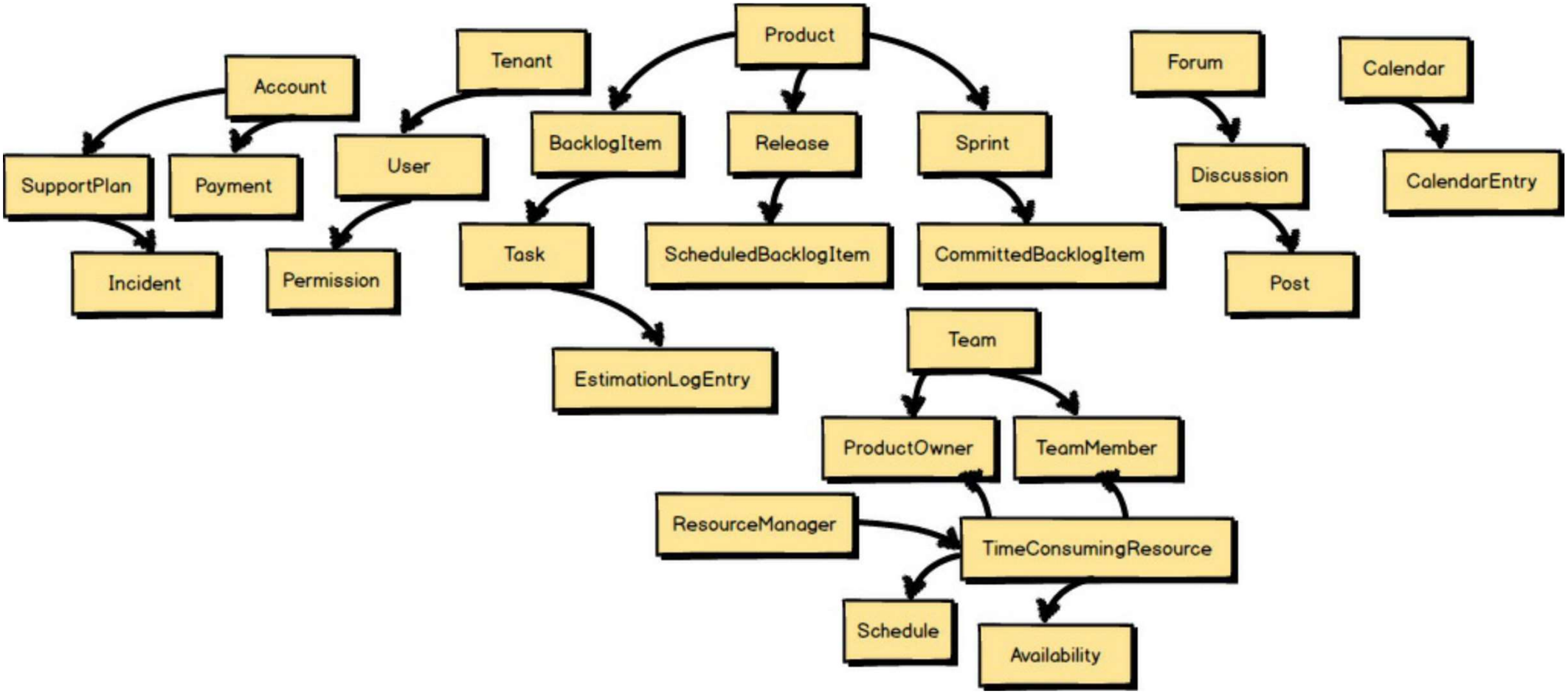
Još zahtjeva …

- ***Trebamo evidenciju korisnika iz različitih organizacija (Tenant) … i kalendare … i forume … i naplatu korisnicima***



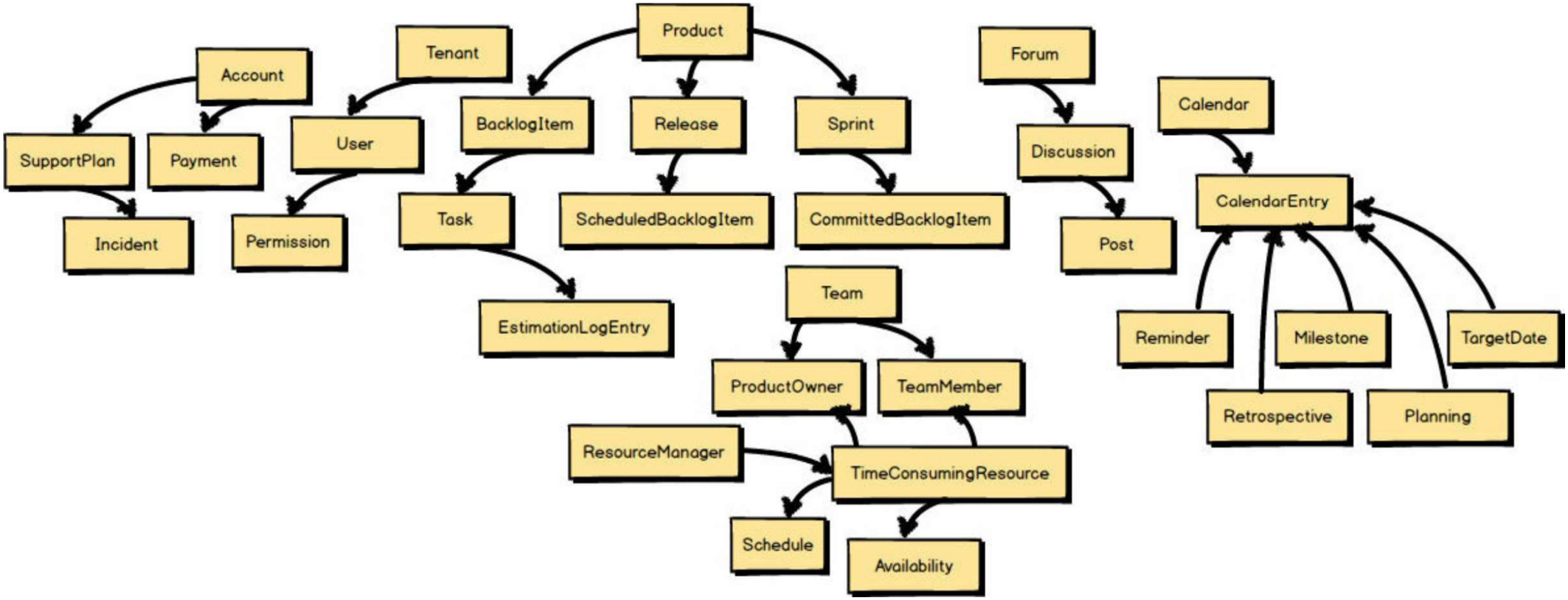
cc BY NC SA

I još uloge članova te njihove raspoloživosti …



BY NC SA

I malo jači kalendar …

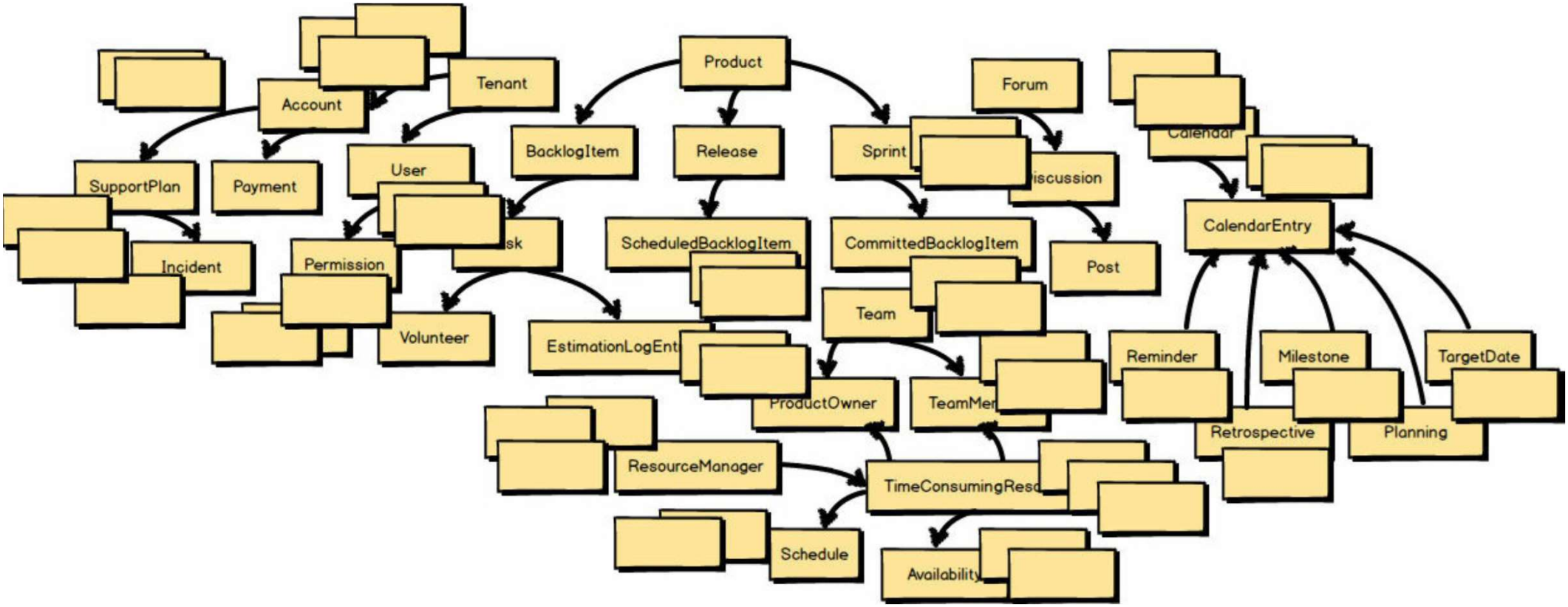


BY NC SA

I još i još …

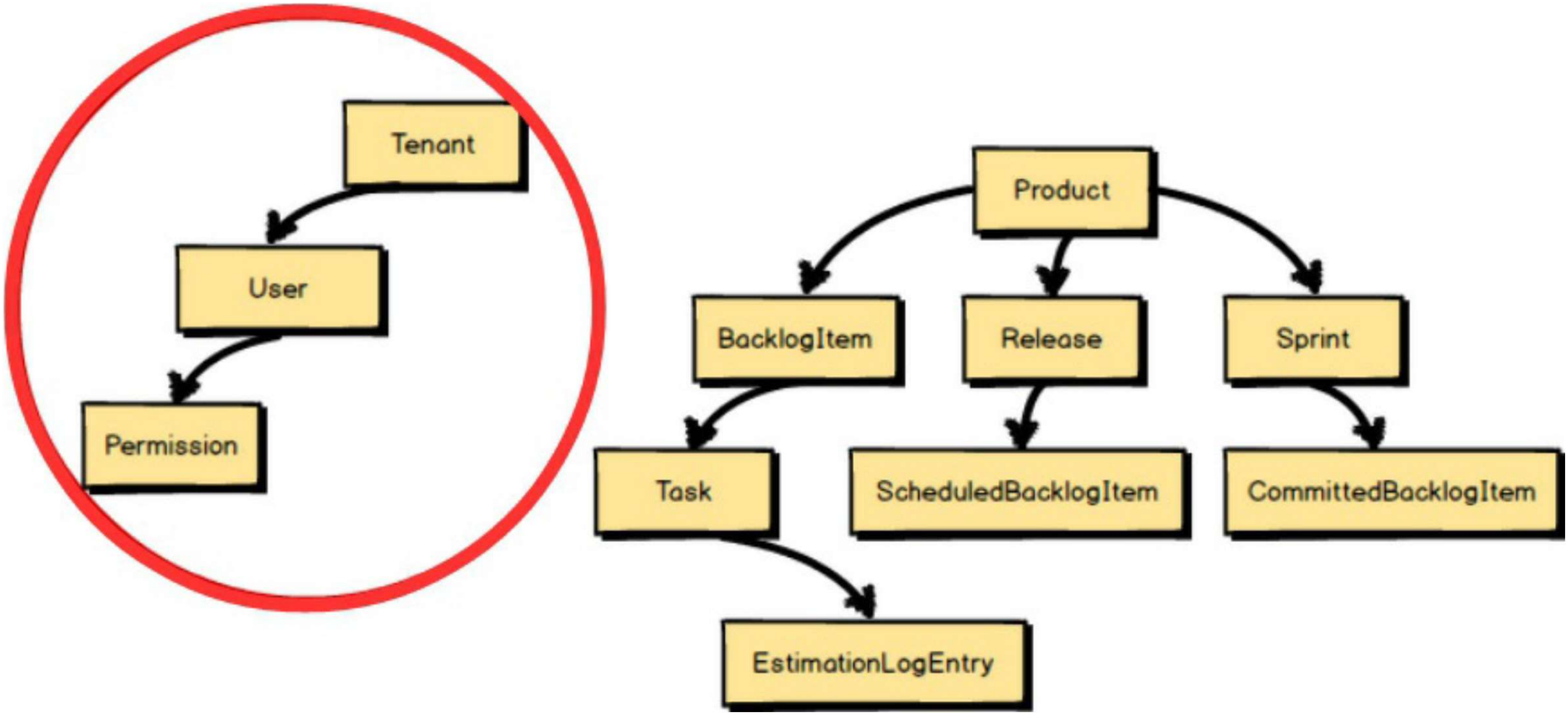
- *Big Ball of Mud*
- *… No Big Design Up Front (jer će se promijeniti)*





Rješenje - korisnici

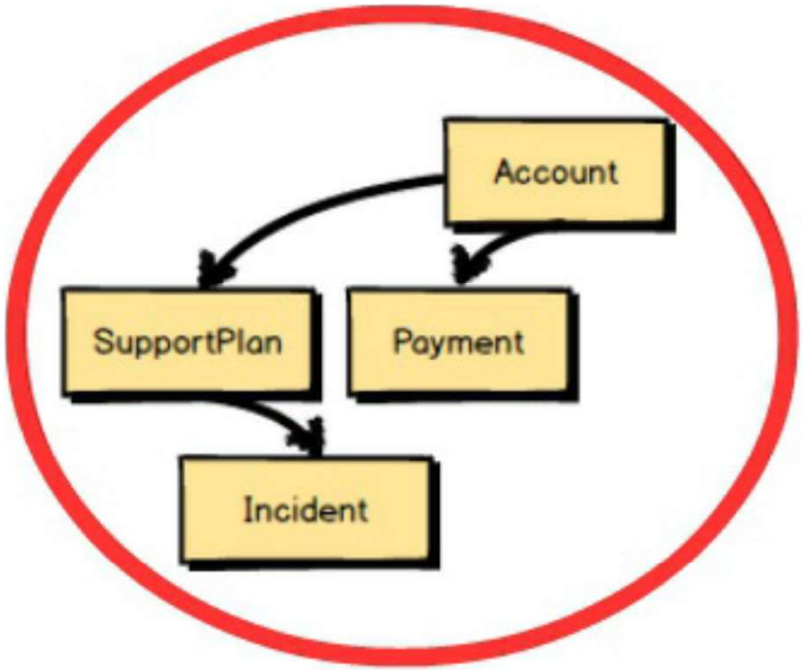
- Nema veze sa Scrumom, odvojiti

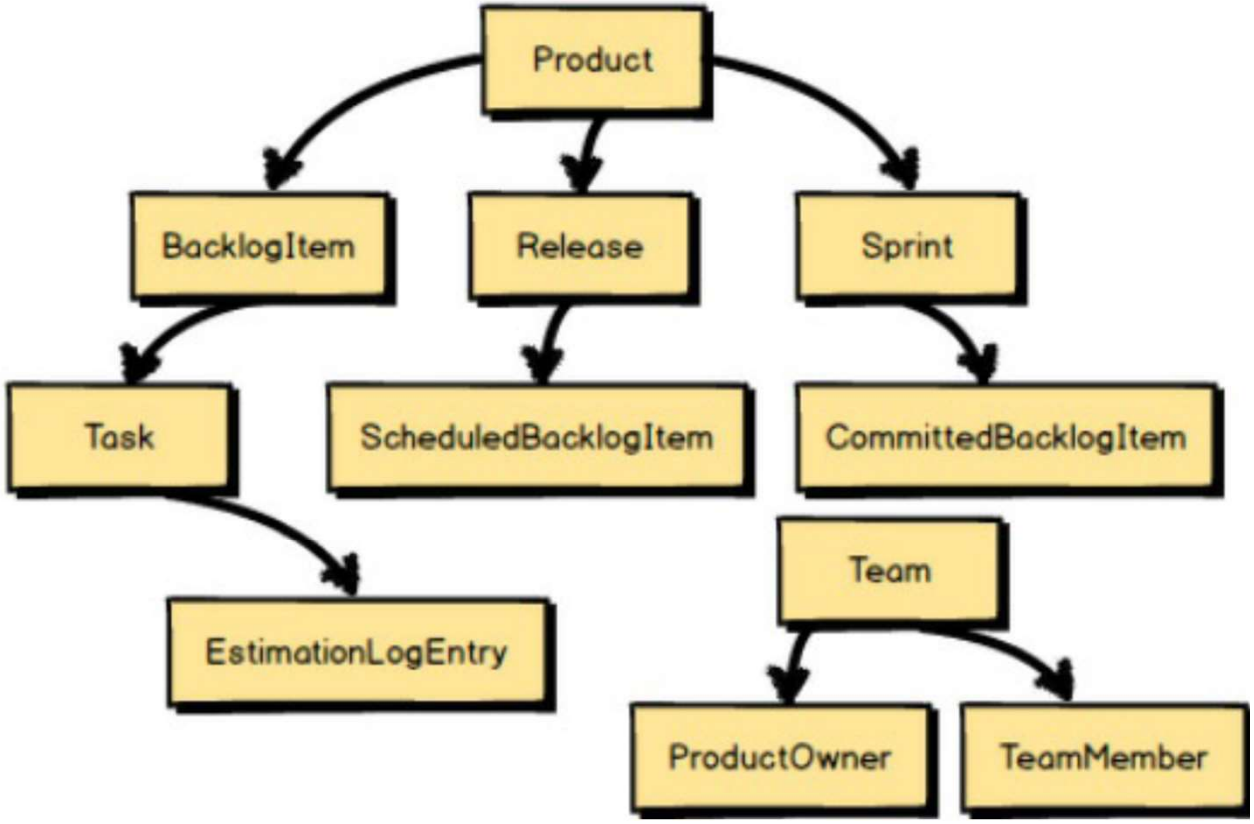


BY NC SA

Rješenje - podrška

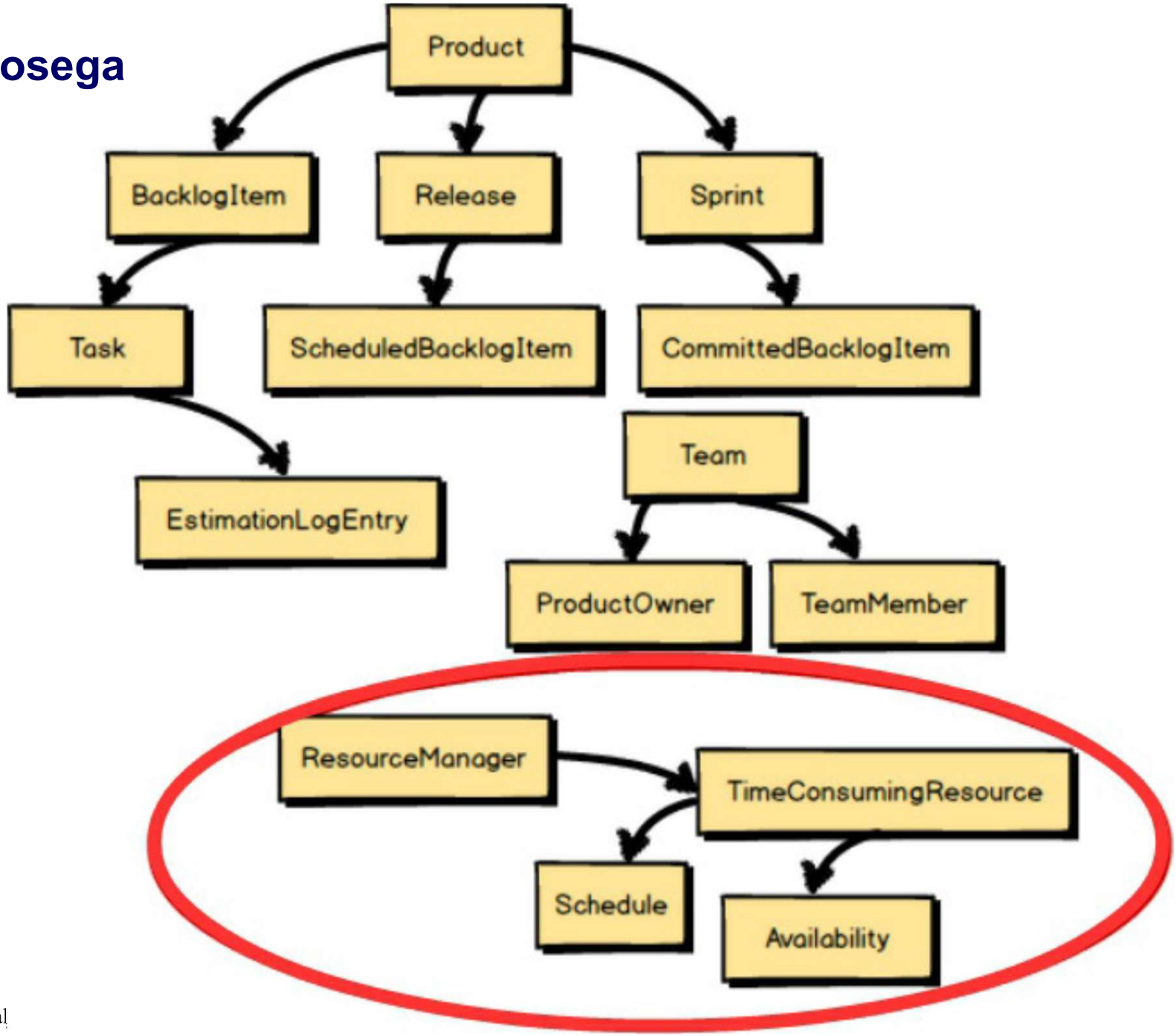
- *Team, ProductOwner i TeamMember* ostaviti jer su dio *Scrum*
- Bavljenje podrškom i naplatom odvojiti





Rješenje - kadroviranje

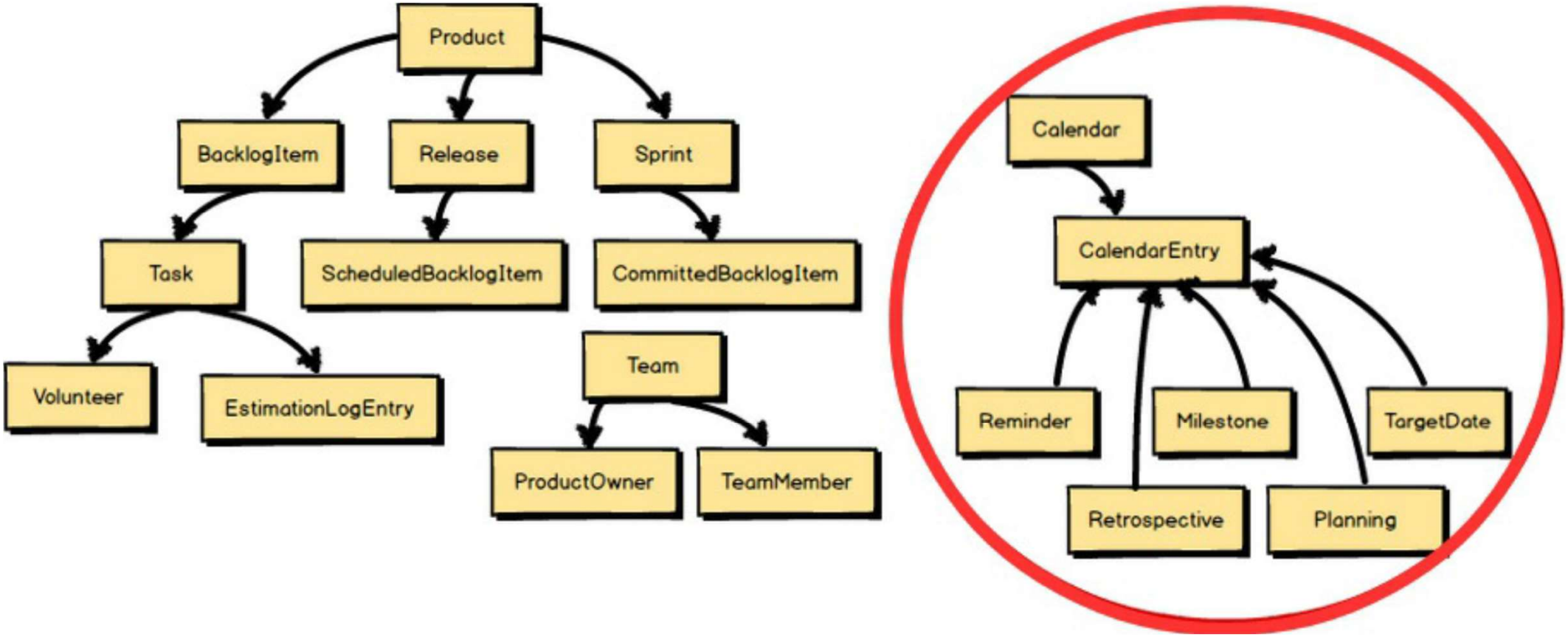
- Izvan dosega



CC BY NC SA

Rješenje – napredni kalendar

- U kontekstu, ali privremeno izvan opsega



cc BY NC SA

Sveprisutni jezik

❑ Scenarij

- The product owner commits a backlog item to a sprint. The backlog item may be committed only if it is already scheduled for release, and if a quorum of team members have approved commitment. If it is already committed to a different sprint, it must be uncommitted first. When the commitment completes, notify the sprint from which it was uncommitted and the sprint to which it is now committed.

❑ Izvršna specifikacija – tehnika *Specification by Example*

- Drugim riječima: **Behavior-Driven Development (BDD)**

```
Scenario:  The product owner commits a backlog item to a sprint
   Given a backlog item that is scheduled for release
   And the product owner of the backlog item
   And a sprint for commitment
   And a quorum of team approval for commitment
   When the product owner commits the backlog item to the sprint
   Then the backlog item is committed to the sprint
   And the backlog item committed event is created
```

Pripadni jedinični test

```
[Test]
public void ShouldCommitBacklogItemToSprint()
{
  // Given
  var backlogItem = BacklogItemScheduledForRelease();

  var productOwner = ProductOwnerOf(backlogItem);

  var sprint = SprintForCommitment();

  var quorum = QuorumOfTeamApproval(backlogItem, sprint);

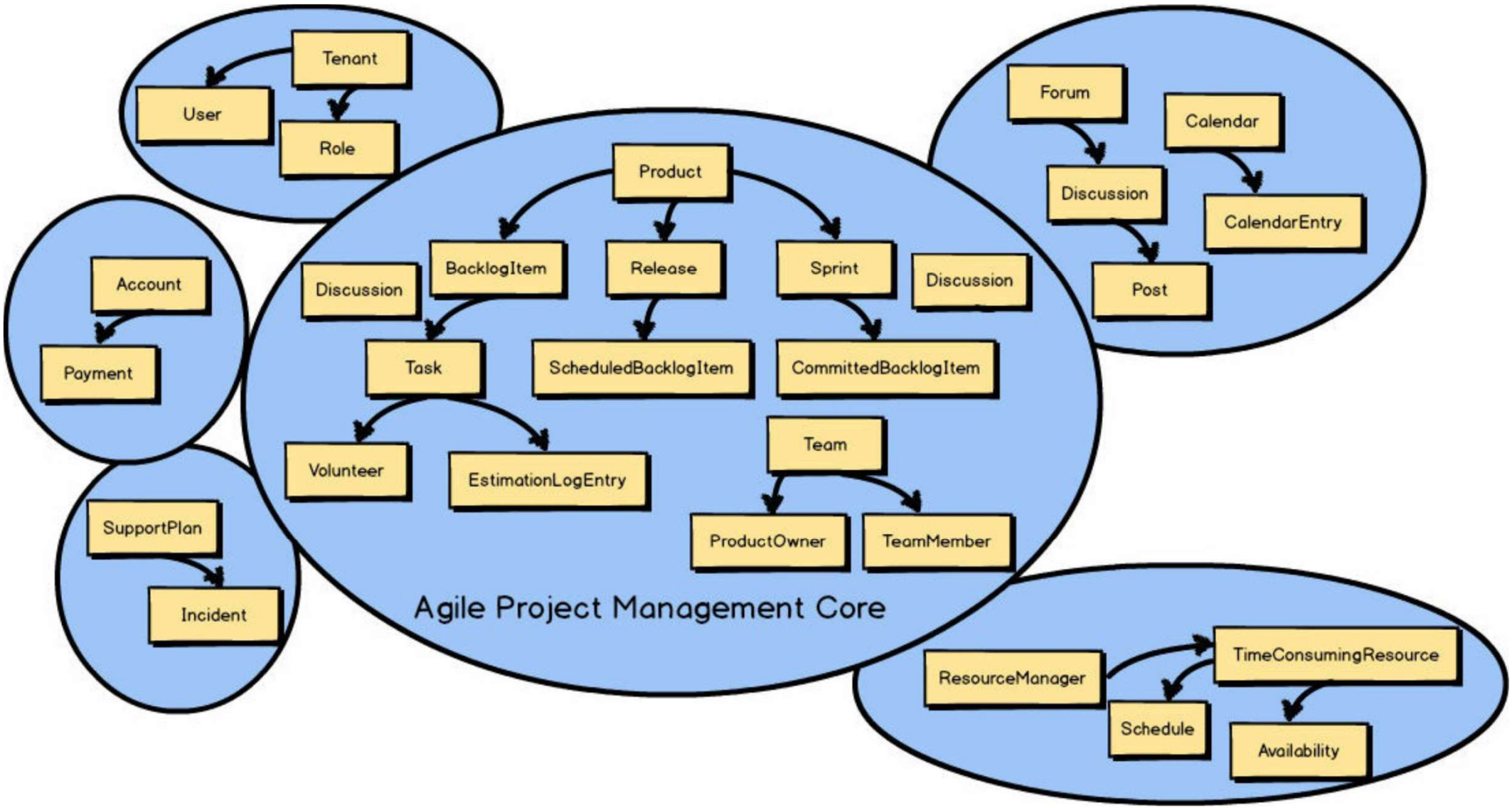
  // When
  backlogItem.CommitTo(sprint, productOwner, quorum);

  // Then
  Assert.IsTrue(backlogItem.IsCommitted());

  var backlogItemCommitted =
    backlogItem.Events.OfType<BacklogItemCommitted>().SingleOrDefault();

  Assert.IsNotNull(backlogItemCommitted);
}
```

Domena i poddomene



Poddomene (*build vs buy vs rent još jedared*!)

- „pojedinačni, logički model područja”

❑ Jezgrena (Core Domain)

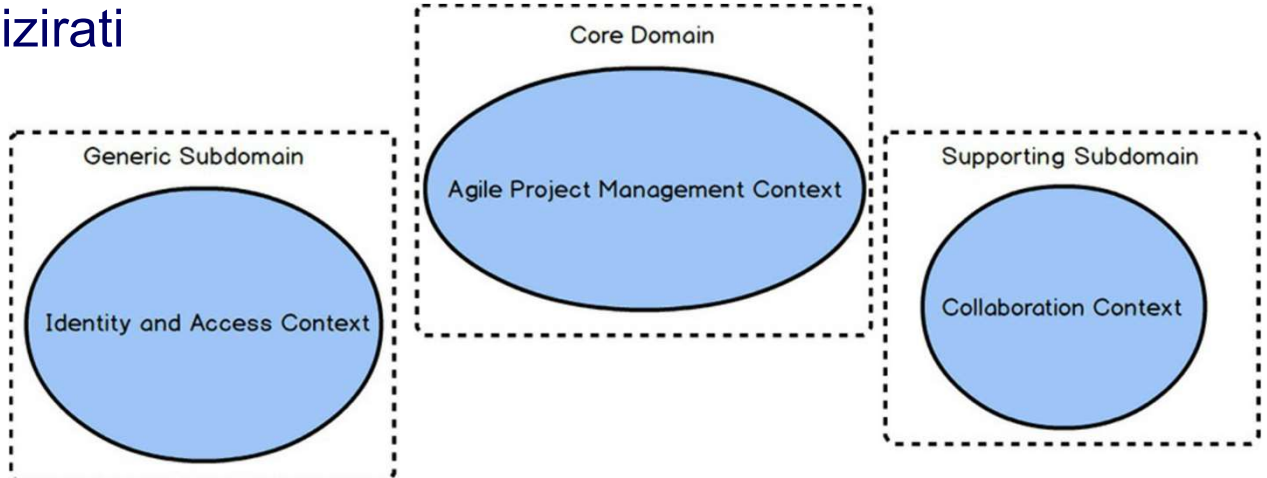
- Dobro definiran model
- Posebnost organizacije - kompetitivna prednost – vlastiti razvoj

❑ Potporna (Supporting Subdomain)

- Zahtijeva razvoj po mjeri
- Ne postoji gotovo rješenje na tržištu – vanjski razvoj (outsourcing)

❑ Opća poddomena (Generic Subdomain)

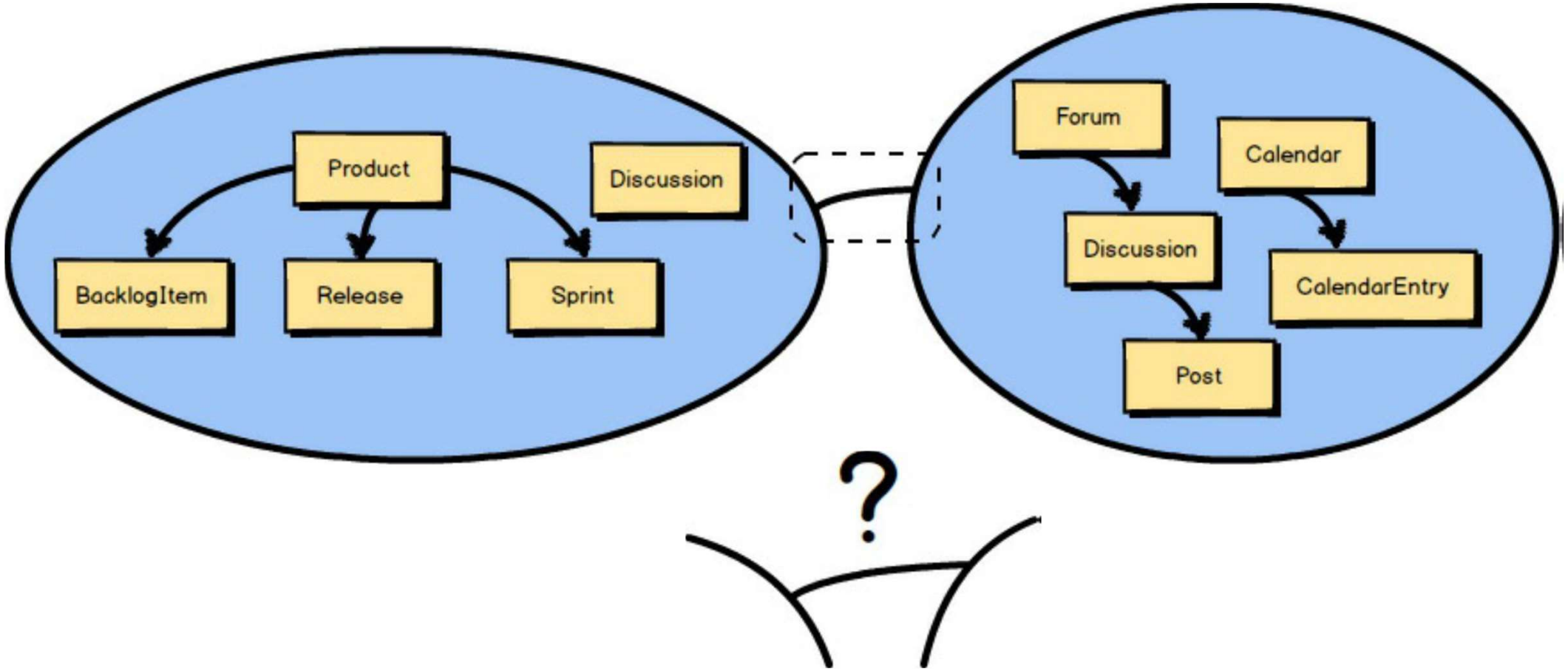
- Kupiti gotovo ili eksternalizirati



CC BY NC SA

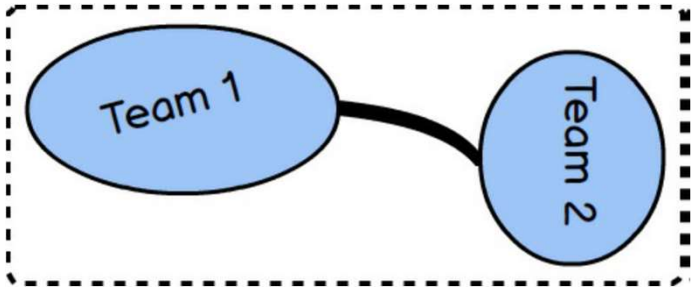
Strateški dizajn mapiranjem konteksta

- Veliki projekti, više ekipa - očuvanje integriteta modela

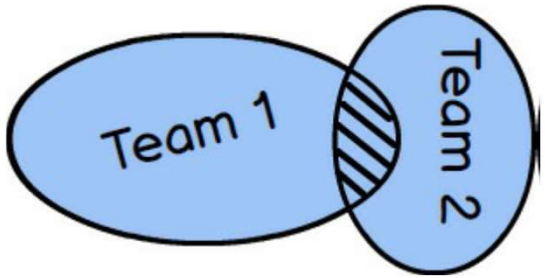


Mapiranje konteksta (2)

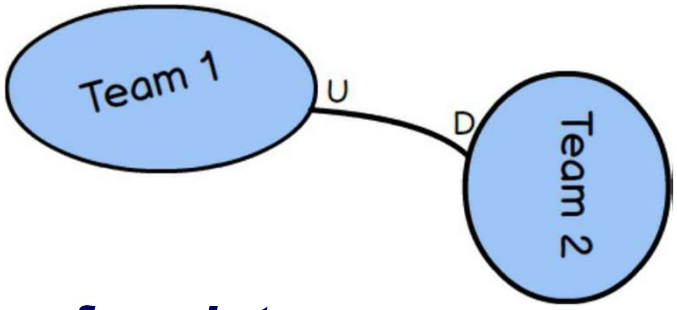
- ***Partnership***
  - Sinkronizacija
  - Kontinuirana integracija



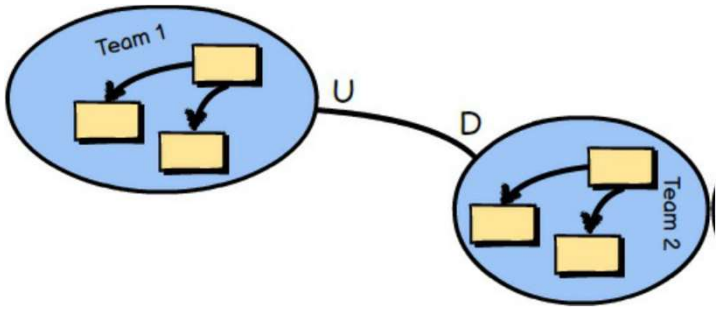
- ***Shared kernel***
  - Dijeljeni dio modela



- ***Customer-Supplier***
  - *Supplier* = *upstream* (U)
    - Određuje kad će i što *Customer* dobiti
  - *Customer* = *downstream* (D)

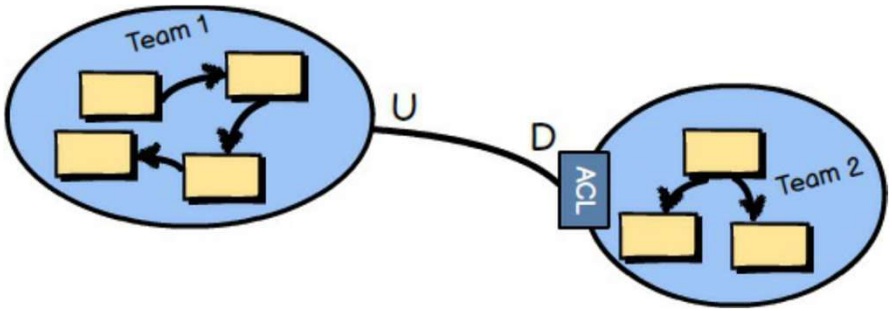


- ***Conformist***
  - Preuzimanje U modela umjesto preslikavanja
  - Pr. kad je dobar (amazon.com)

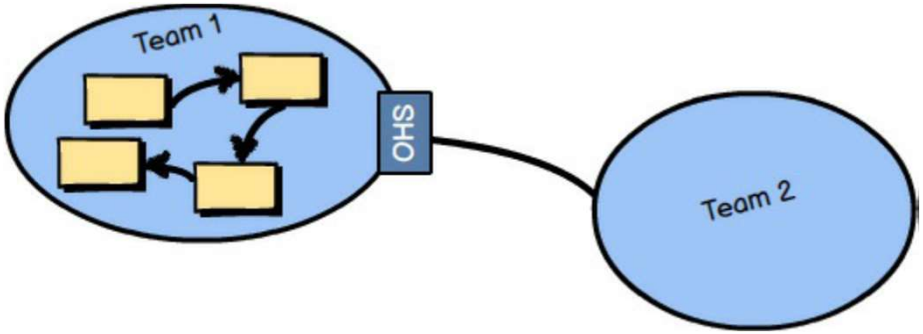


Mapiranje konteksta (3)

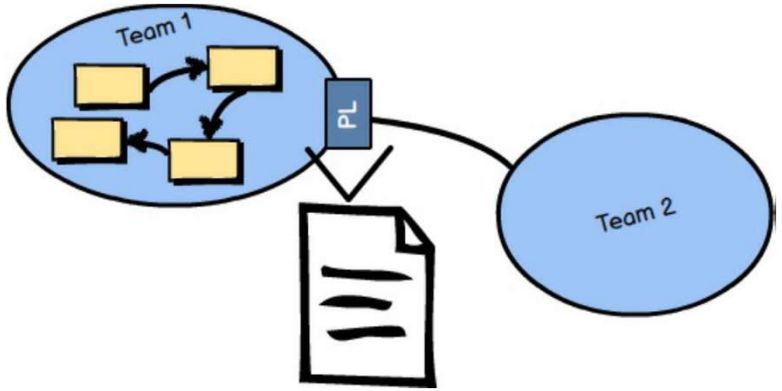
- ***Anticorruption Layer***
  - Translacija modela u prikladni



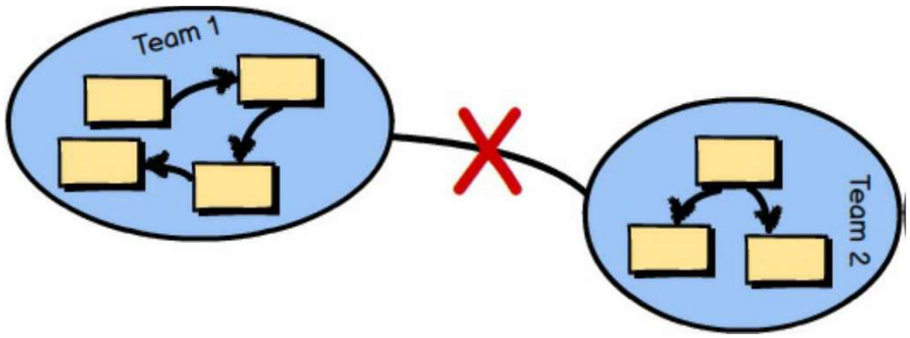
- ***Open Host Service***
  - OHS definira protokol ili sučelje
  - Otvoren i lagan za korištenje – provocira konformista



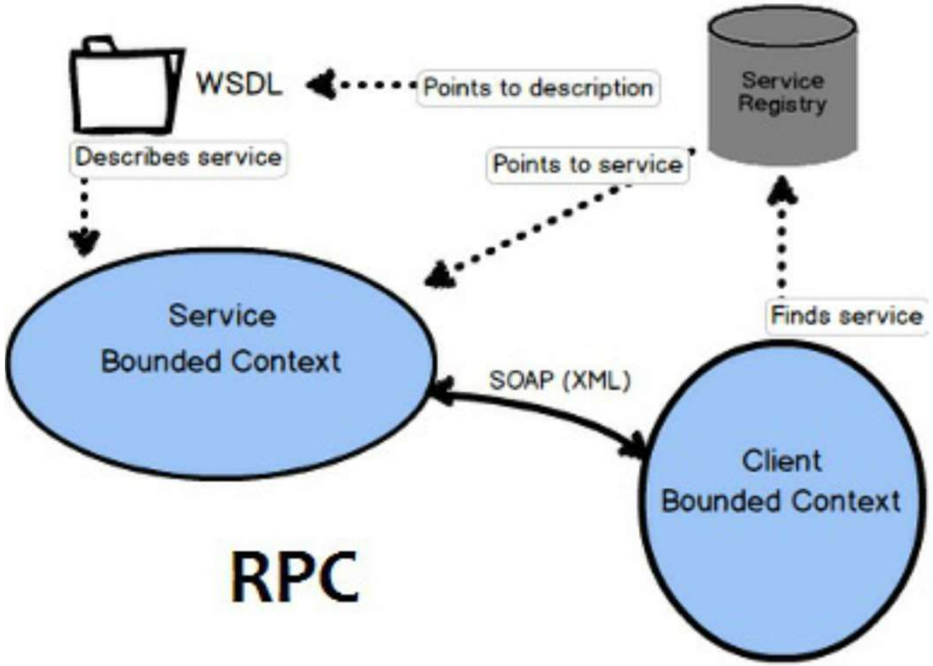
- ***Published Language***
  - Dobro dokumentirani jezik razmjene
  - Npr. XML schema, JSON schema

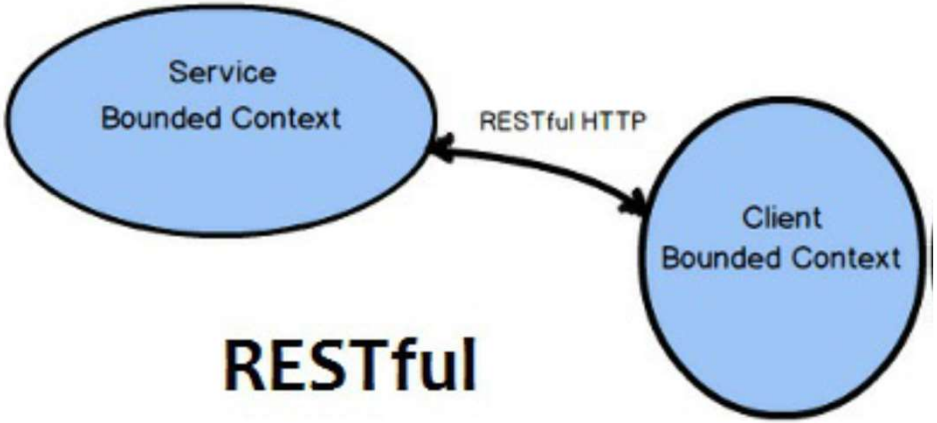


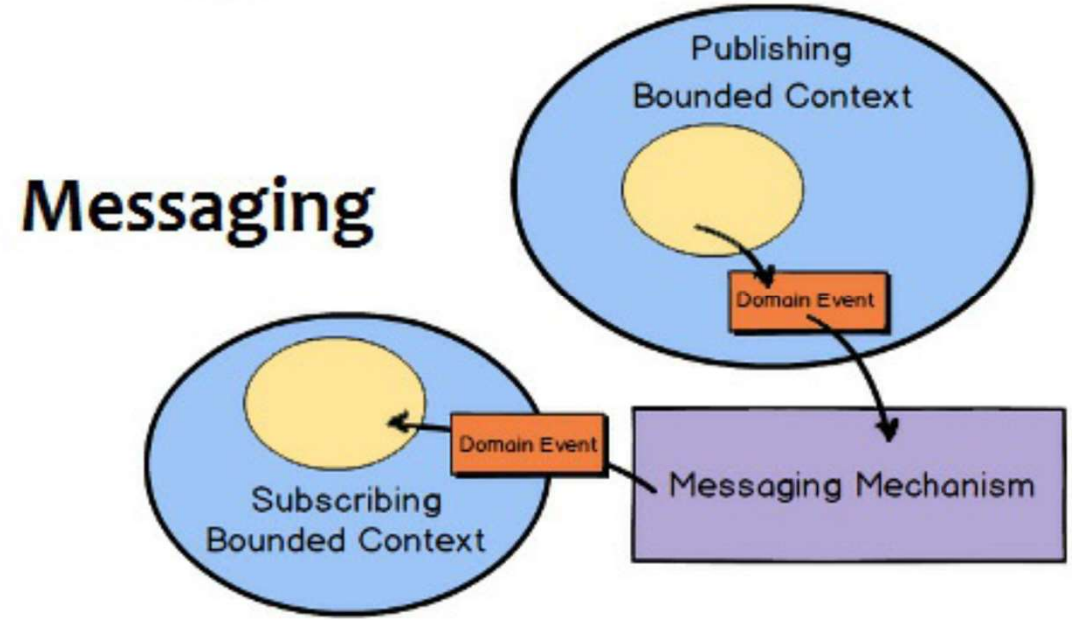
- ***Separate Ways***
  - Vlastito specijalizirano rješenje
  - Za tu domenu, ne nužno „uvijek”



Mehanizmi mapiranja (o arhitekturi drugi put)

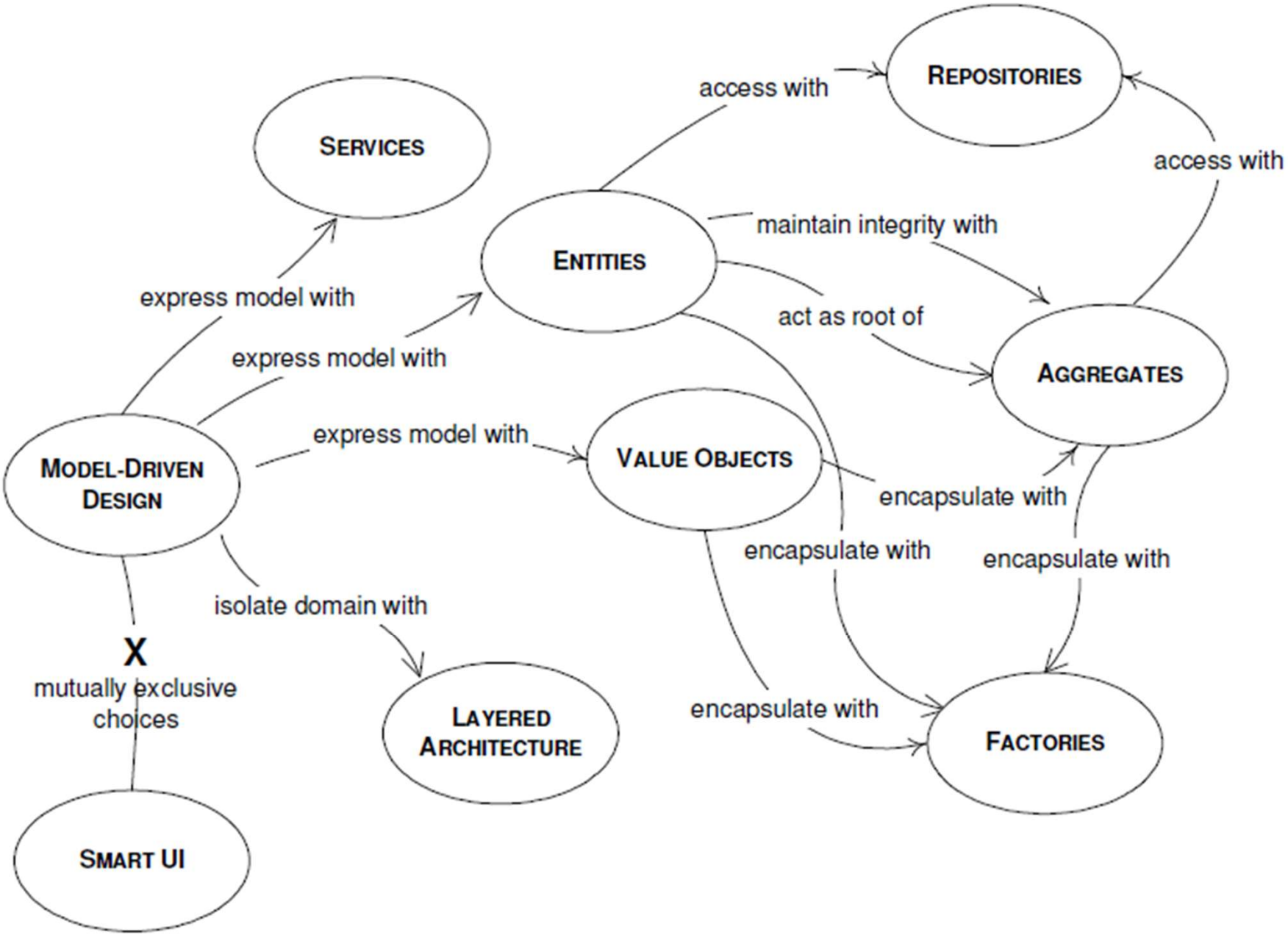






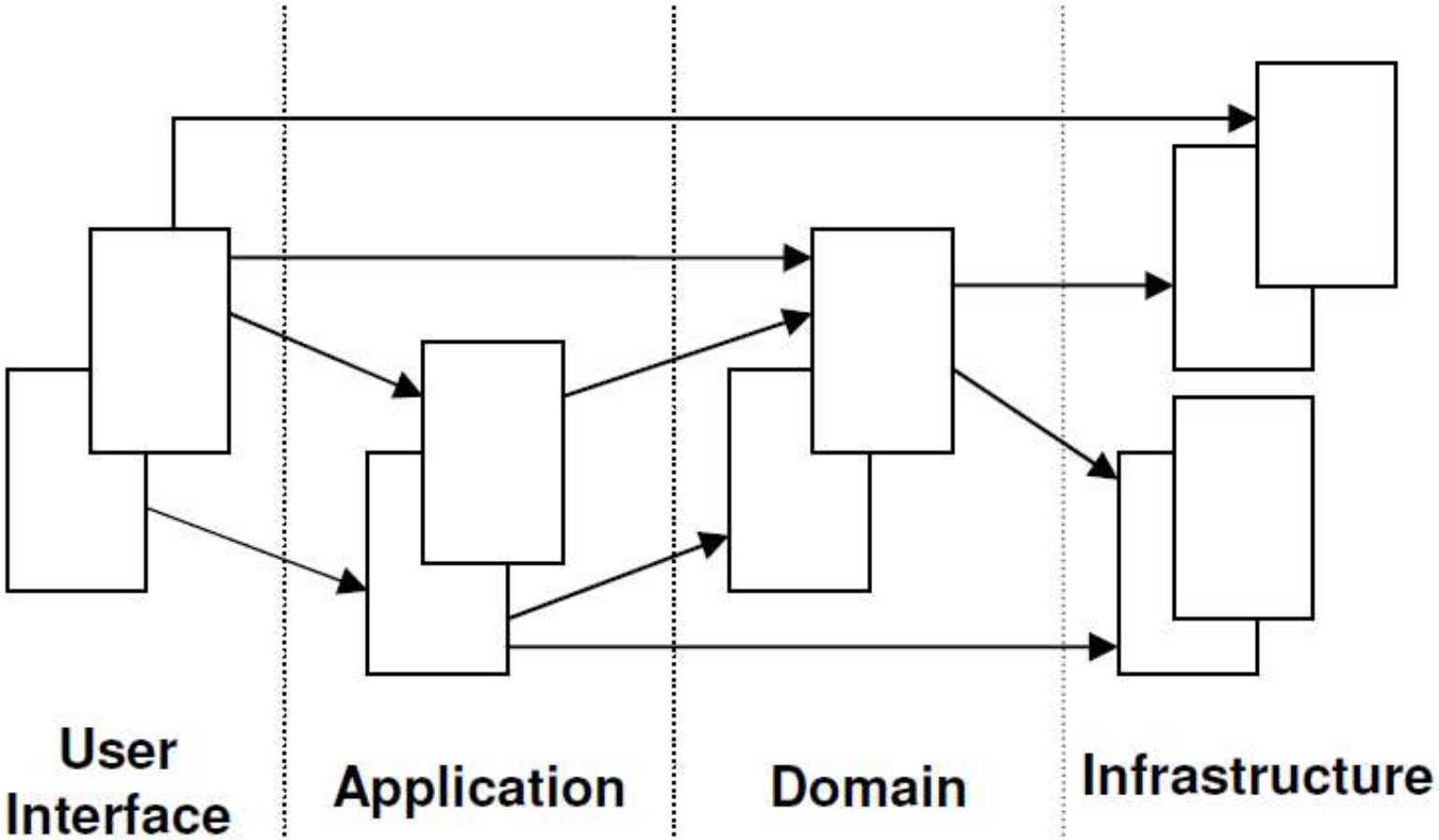
CC BY NC SA

Građevni blokovi modelom vođenog dizajna (MDD)



Slojevita arhitektura

- **Slojevi**
  - Korisničko sučelje – prezentacija
  - Aplikacijski – koordinacija aplikacijskih aktivnosti
  - Domena – stanje poslovnih objekata, poslovna logika
  - Potporna knjižnica - komunikacija slojeva, implementacija pohrane



cc BY NC SA

Entiteti i vrijednosni objekti

- **Entiteti (entities) – business entity (BE)**
  - Imaju identitet koji se ne mijenja promjenom stanja softvera
  - Atributi entiteta se mogu mijenjati (*mutable*)
  - Imaju ponašanje - poslovna logika

- **Vrijednosni objekti (value objects - VO)**
  - „stvari” bez identiteta, jedinstvenosti
  - Jednaki ako imaju jednake vrijednosti svih atributa
  - Atributi često nepromjenjivi (*immutable*) – uništiš pa stvoriš drugi
  - Varijanta – **Data Transfer Object (DTO)**
    - Klasa podataka bez logike
  - Izvorno: **Plain Old Simple Class Object (POCO)**
    - .NET: Plain Old Simple CLR Object
    - Java: Plain Old Simple Java Object (POJO)

```
// Entity
public class Person
{
    public int Id { get; set; }
    public string Name { get; set; }
    public Address Address { get; set; }
}

// Value Object
public class Address
{
    public string City { get; set; }
    public string ZipCode { get; set; }
}
```

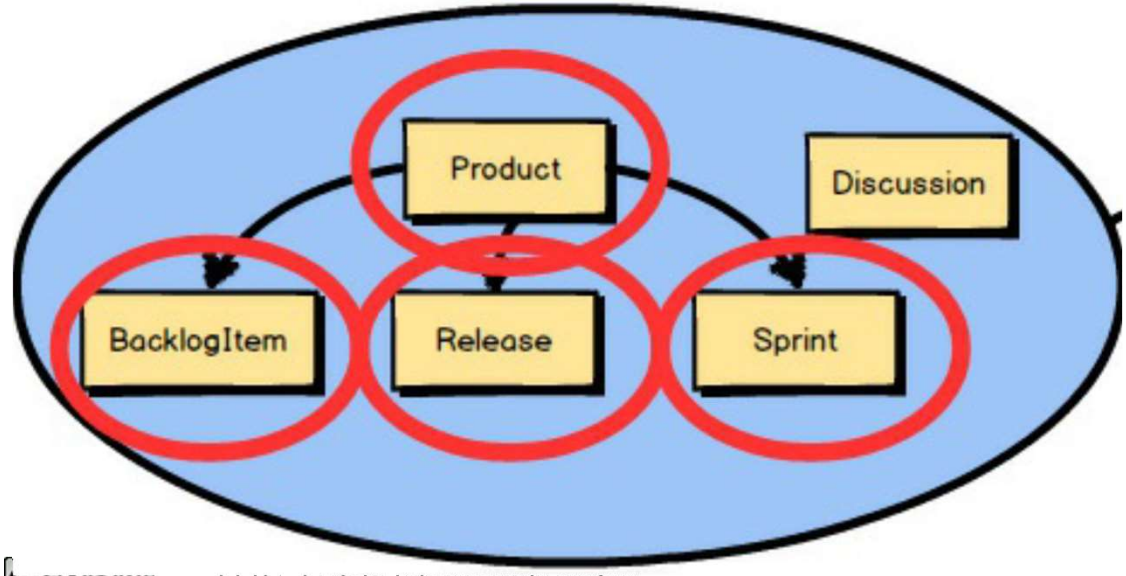
BY NC SA

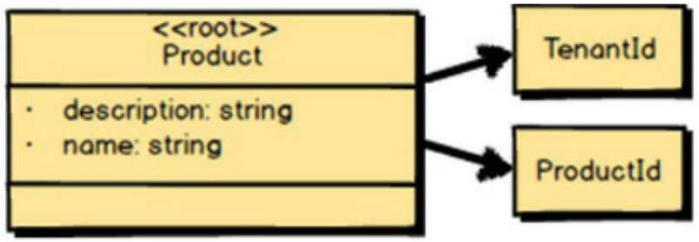
Agregati

- Grupe entiteta i vrijednosti, tretirani kao cjelina
- Svaki agregat ima jedan korijenski entitet (root)
- Identitet korijena je globalan, identiteti unutarnjih entiteta lokalni
- Vanjski objekti mogu referencirati samo korijen
- Interni objekti ne mogu biti promijenjeni izvan agregata
- Svaki agregat treba biti izmijenjen i pohranjen u zasebnoj transakciji

❑ **Primjer:**

- Lijevo - svaki zaokruženi je agregat, osim diskusije koja je VO
- Desno – agregat *Product*, kompozitnog identiteta 2 VO, bez *gettera/settera*



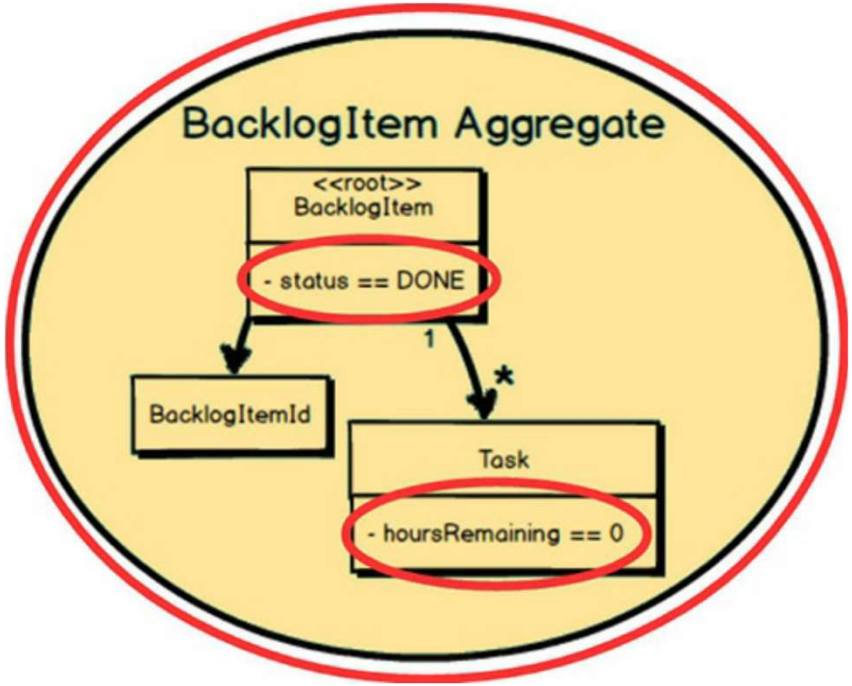



```
public class Product : Entity
{
  private string description;
  private string name;
  private ProductId productId;
  private TenantId tenantId;
}
```

Taktički dizajn agregatima – osnovna pravila (1)

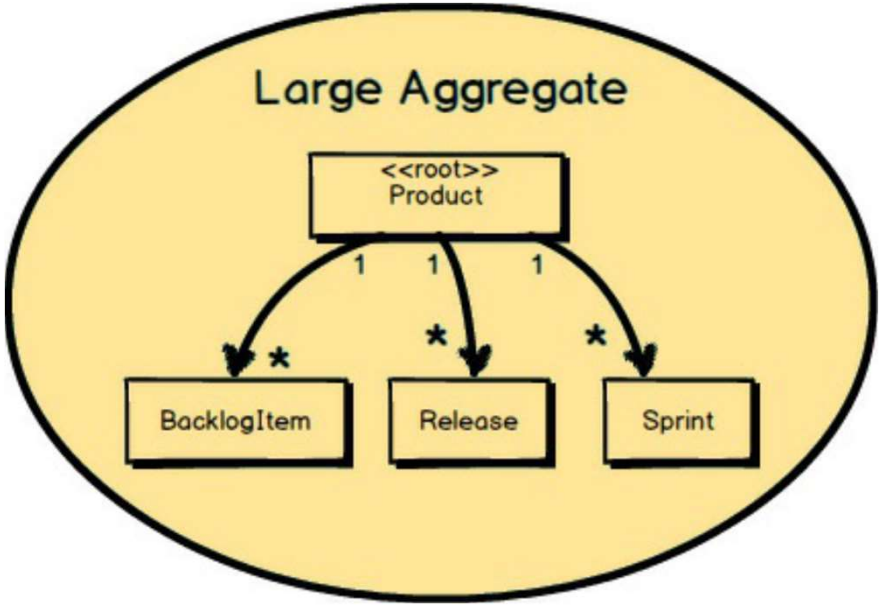
- ***Protect business invariants inside Aggregate boundaries***
  - *Invarijanta = Karakteristika nepromjenjivosti*
    https://www.enciklopedija.hr/natuknica.aspx?ID=27707

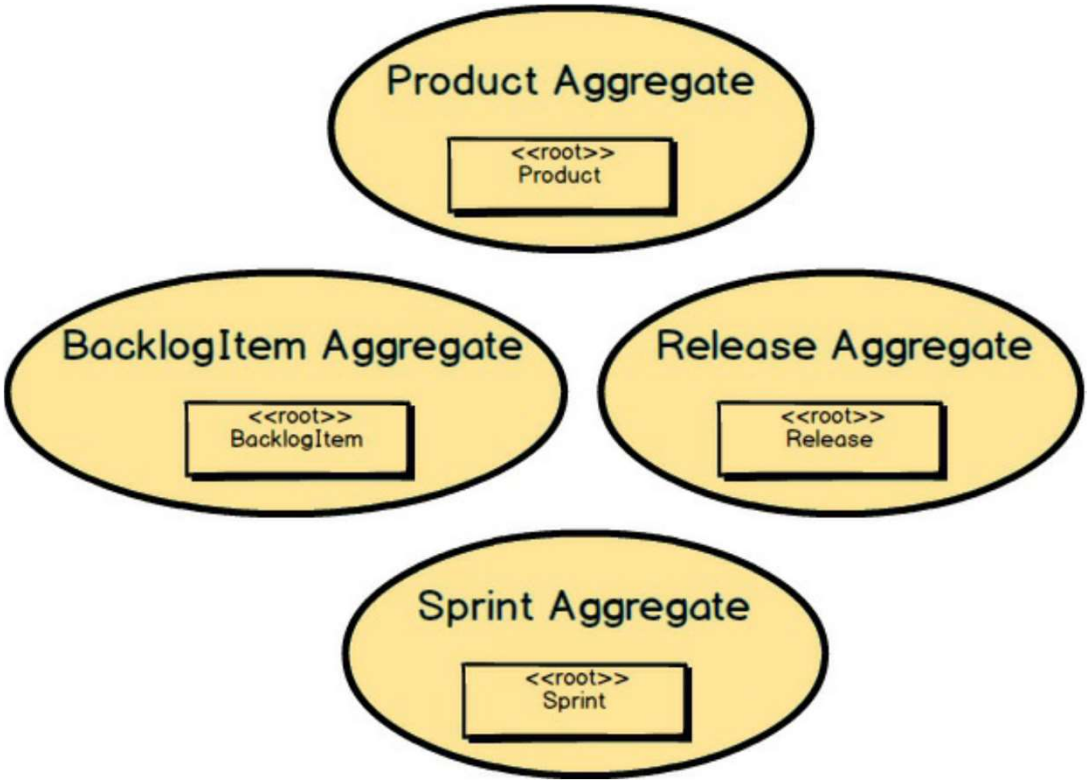
- **U našem slučaju konzistentnost**
  - „po završetku transakcije svi zavisni dijelovi moraju biti konzistentni u odnosu na korijen”
  - Pr: kad ponestane posla na zadacima, zaostatak posla bude gotov (DONE)



Taktički dizajn agregatima – osnovna pravila (2)

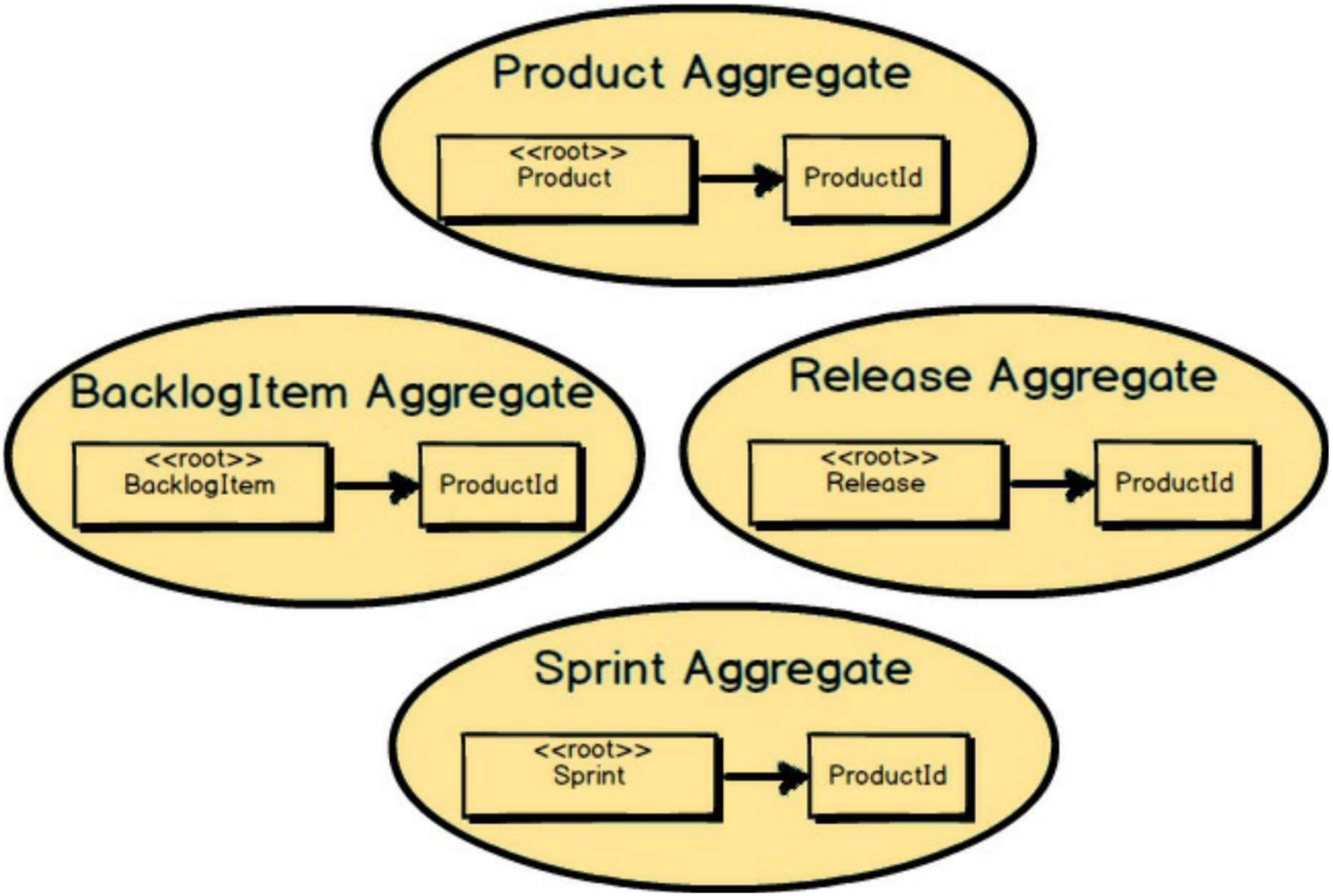
- ***Design small Aggregates***
  - Manje je lakše savladivo
  - Načelo jednostruke odgovornosti (pogledati Responsibility Driven Design)





Taktički dizajn agregatima – osnovna pravila (3)

- ***Reference Other Aggregates by Identity Only***
  - Referencirati druge agregate samo identitetom

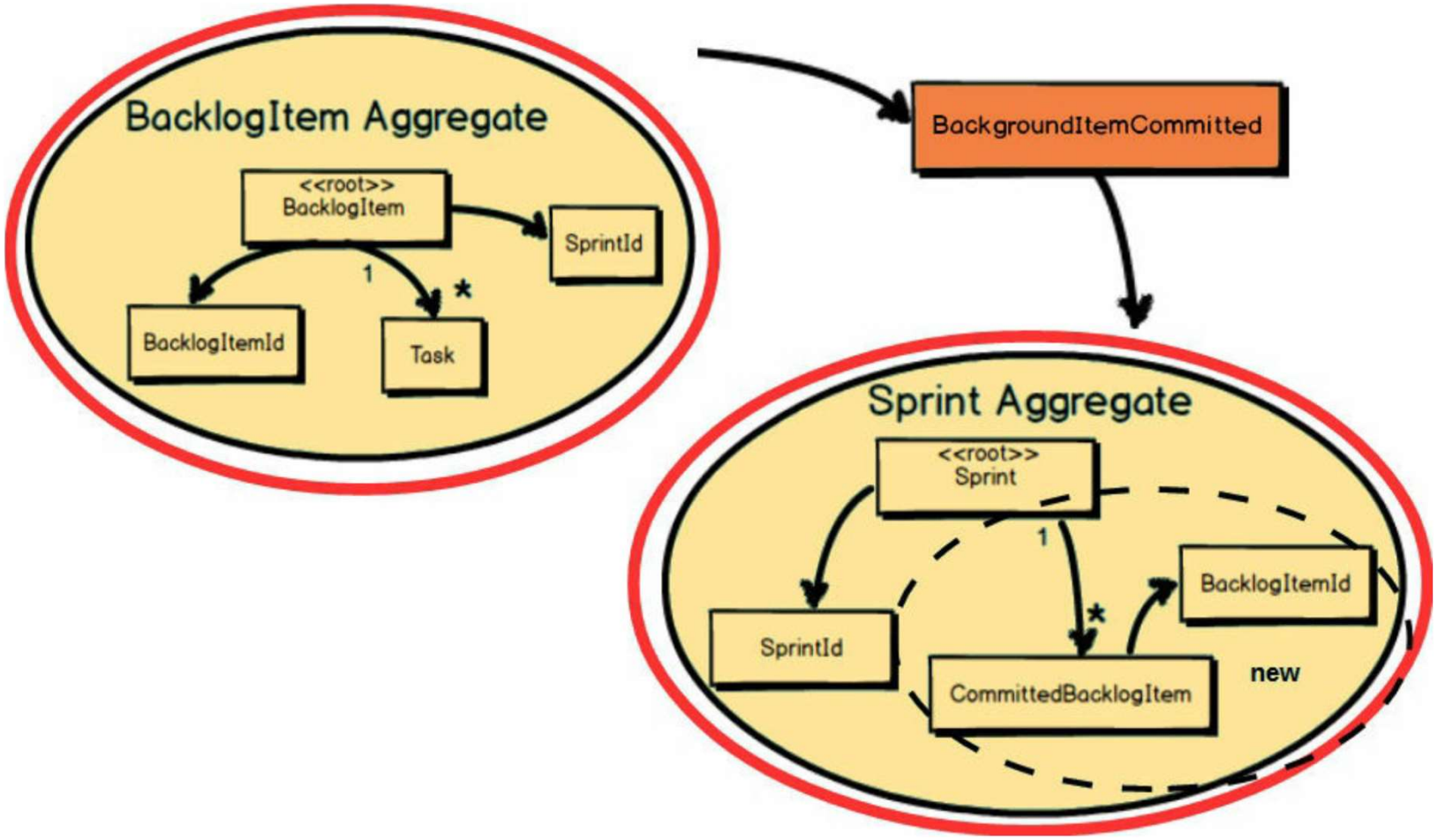


cc BY NC SA

Taktički dizajn agregatima – osnovna pravila (4)

- ***Update Other Aggregates Using Eventual Consistency***
  - *Konačna konzistentnost*

- **Primjer: završetak preostalog posla posljedično ažurira sprint**



Usluge (Services)

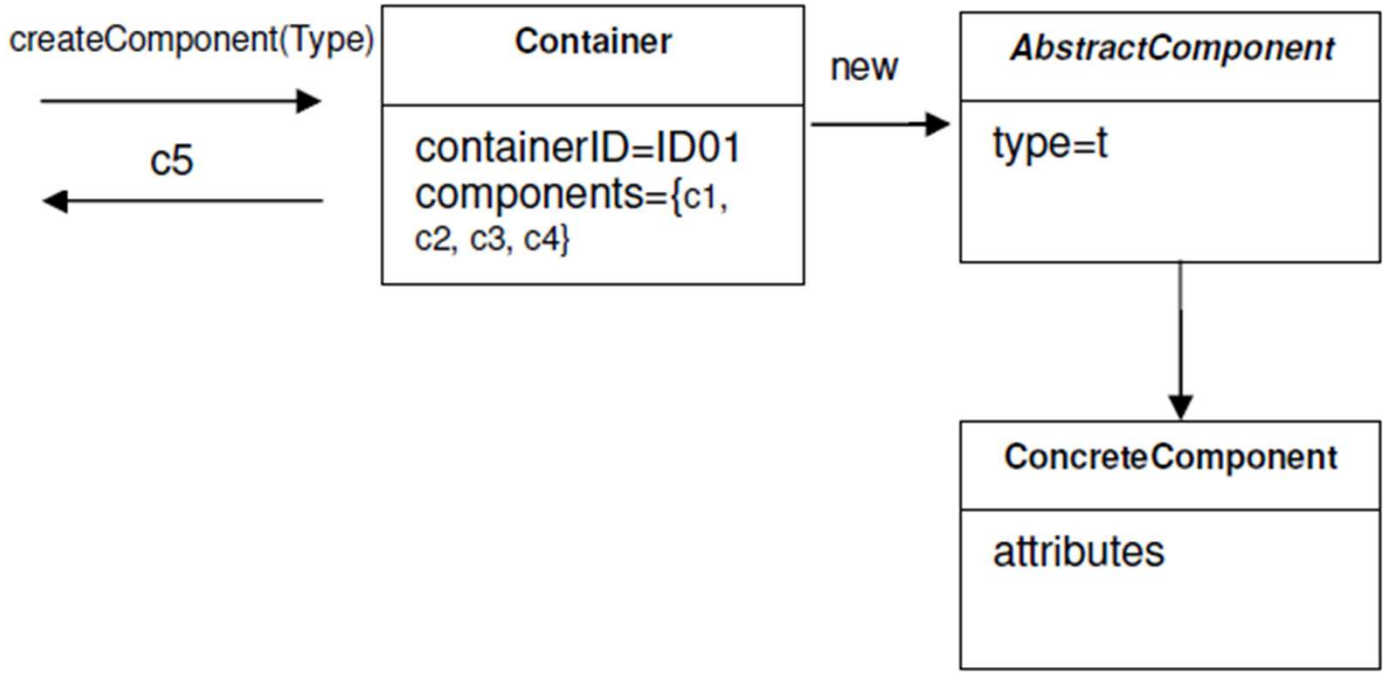
- **Servisi mogu postojati u različitim slojevima**
  - Aplikacijska usluga
    - „bankarska aplikacija može izvesti transakcije u proračunsku tablicu”
  - Domenska usluga
    - „knjiženje uplata i isplata po računu”
  - Infrastrukturna usluga
    - „sučelje koje učahuruje email sustav za slanje poruka o promjeni stanja”

- **Domenska usluga**
  - Poslovna logika domene koja nije dio entiteta ili vrijednosnog objekta
  - Obično rukuje s više entiteta i vrijednosnih objekata
  - Nema stanje! (Services are stateless!)
  - Izložena kao sučelje definirano modelom
  - Parametri i rezultati su domenski objekti

cc BY NC SA

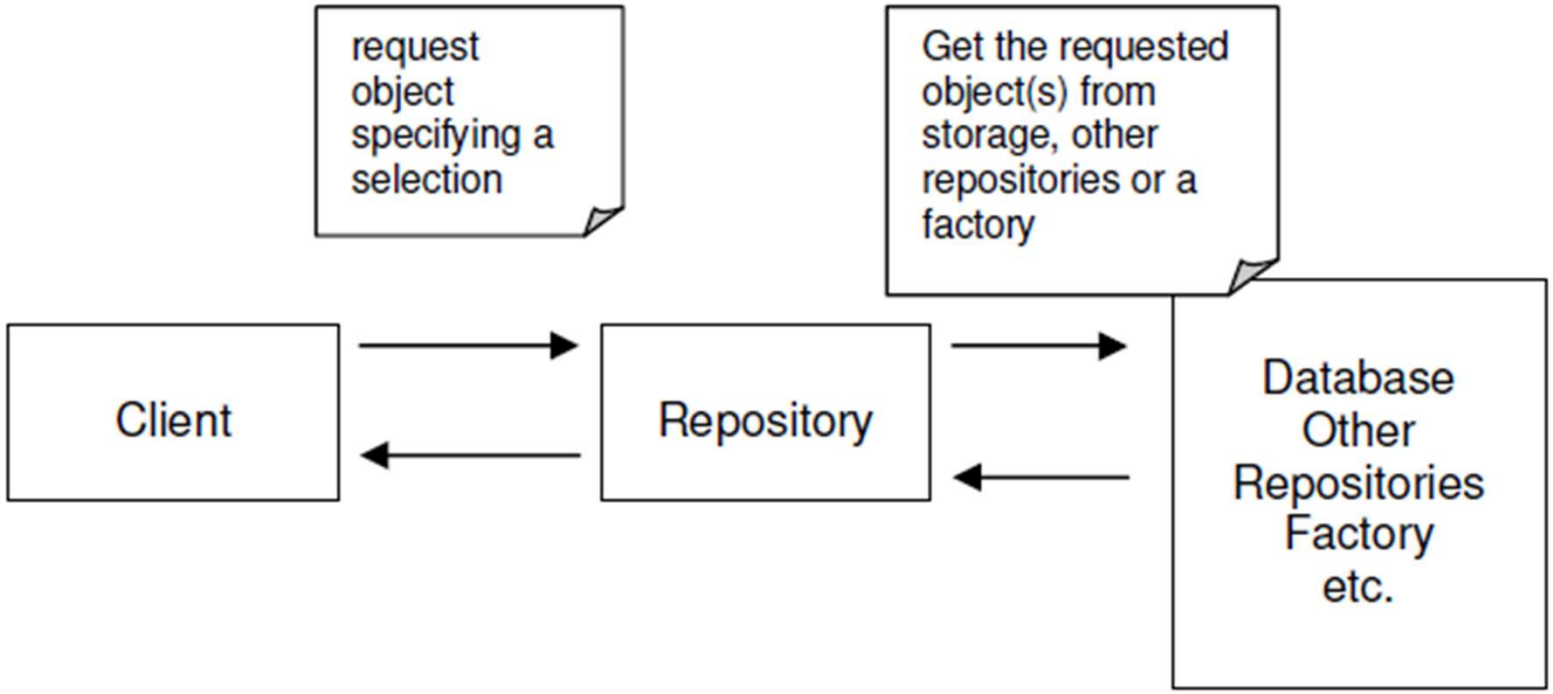
Tvornice (Factories)

- Objekt koji kreira druge objekte
- Kreira i upravlja složenim domenskim objektima
- Posebno koristan za stvaranje agregata

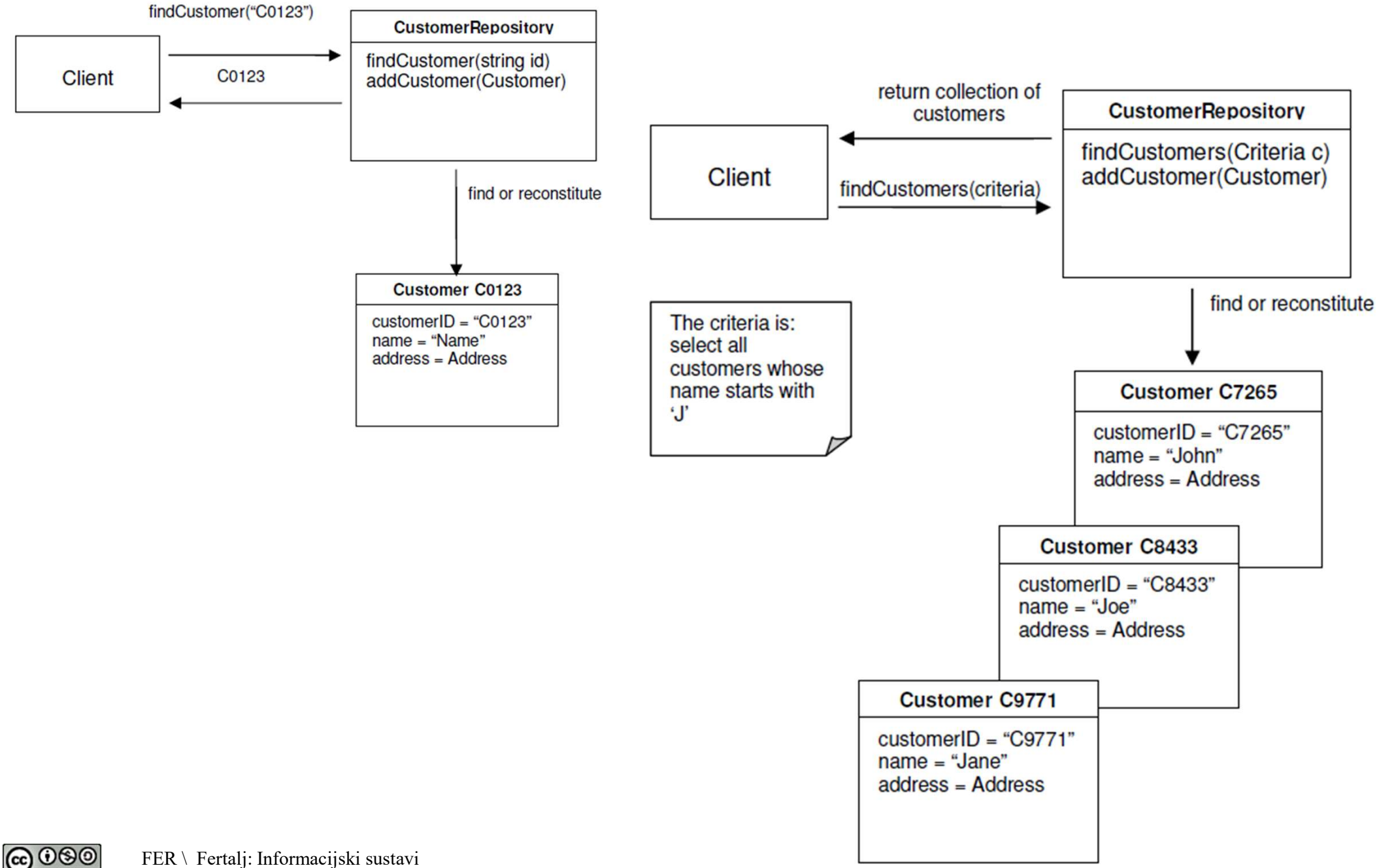


Repozitoriji (Repositories)

- Repozitorij učahuruje pohranu i dohvat domenskih objekata (persistence and retrieval)
- Čisto razdvajanje i jednosmjerna zavisnost između domene i sloja za preslikavanje podataka
- Jedan po agregatu!



Primjer: dohvat jednog, dohvat više



Tvornice i repozitoriji

- Zasebne uloge, jedan stvara, drugi sprema i pronalazi

